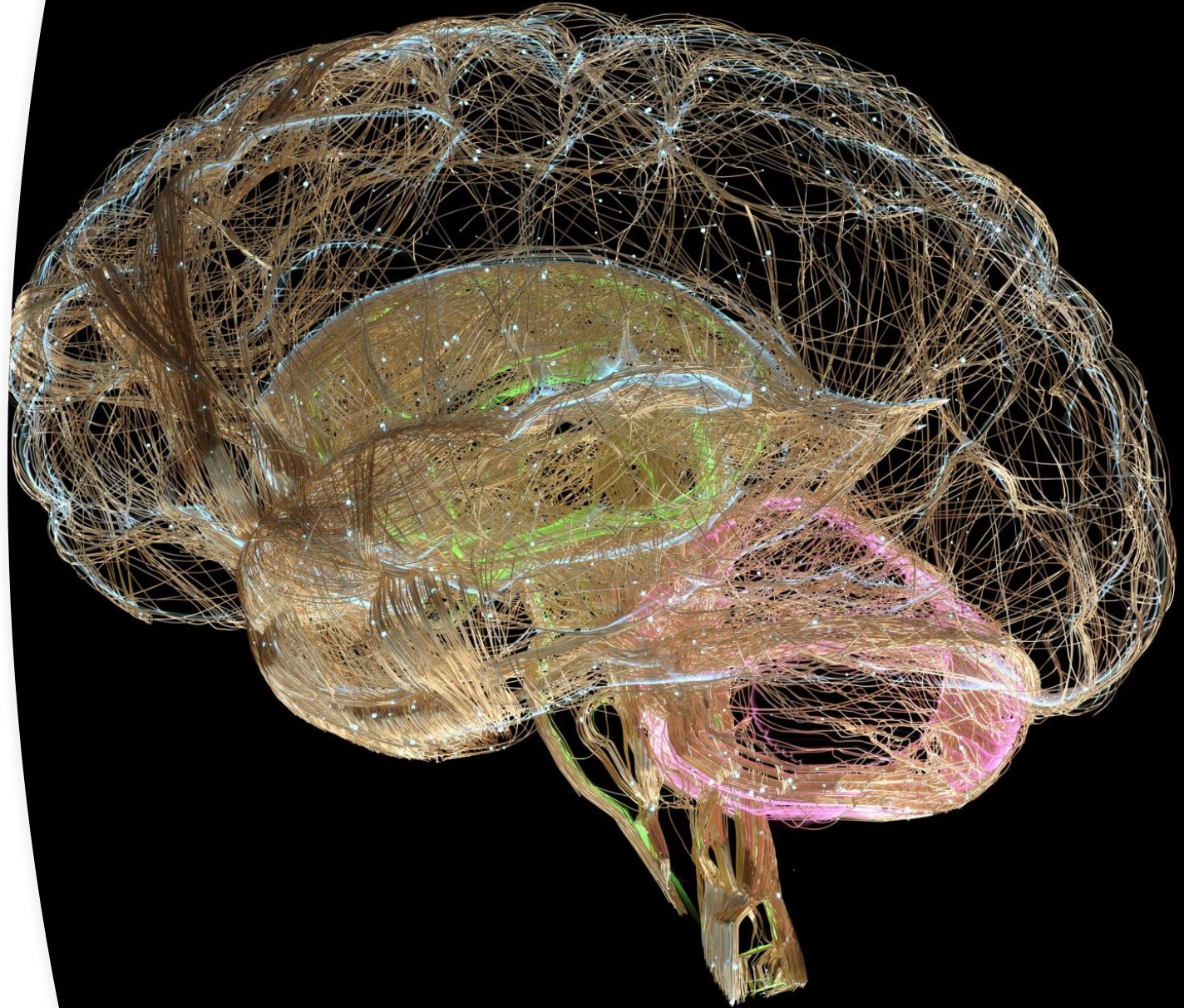
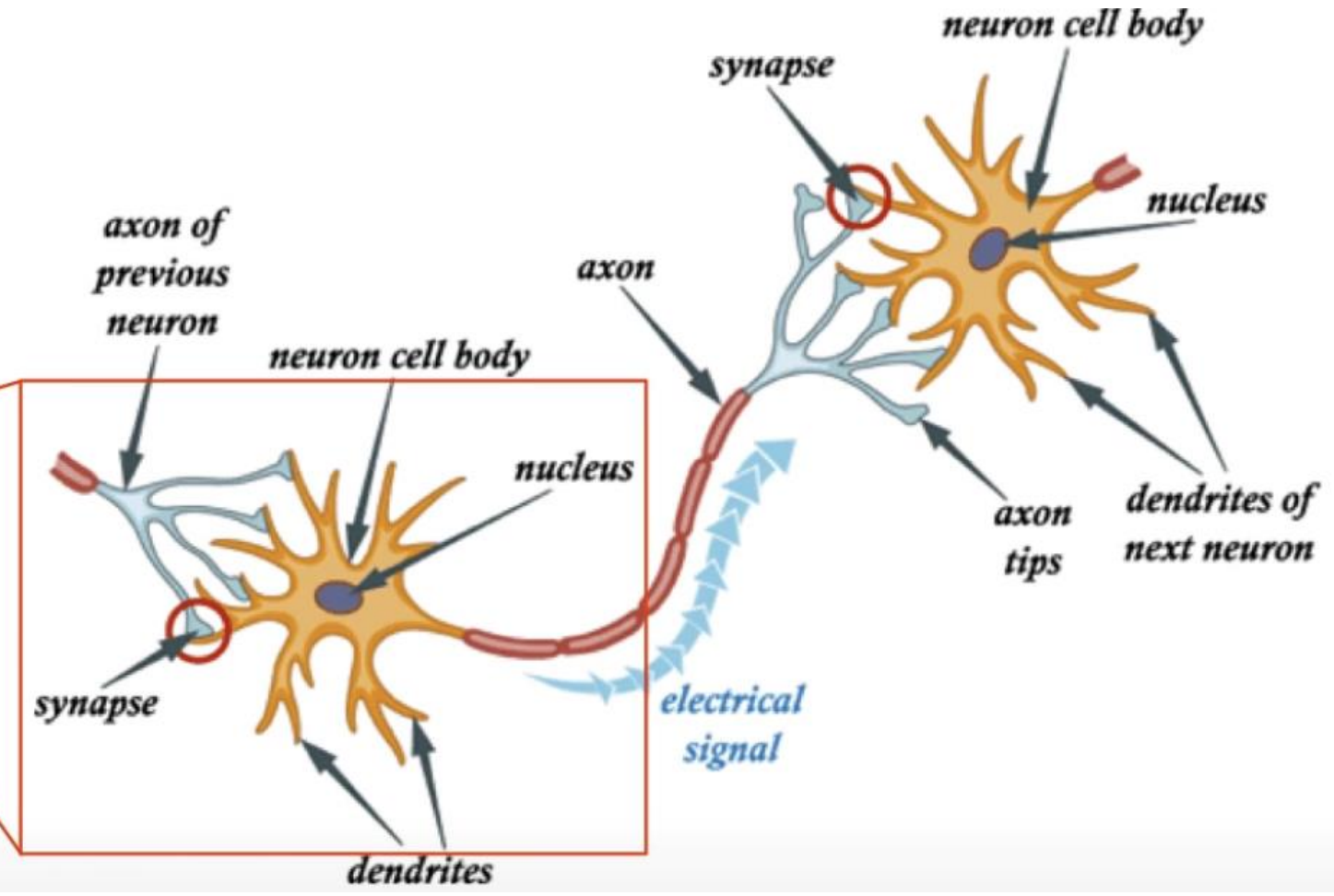
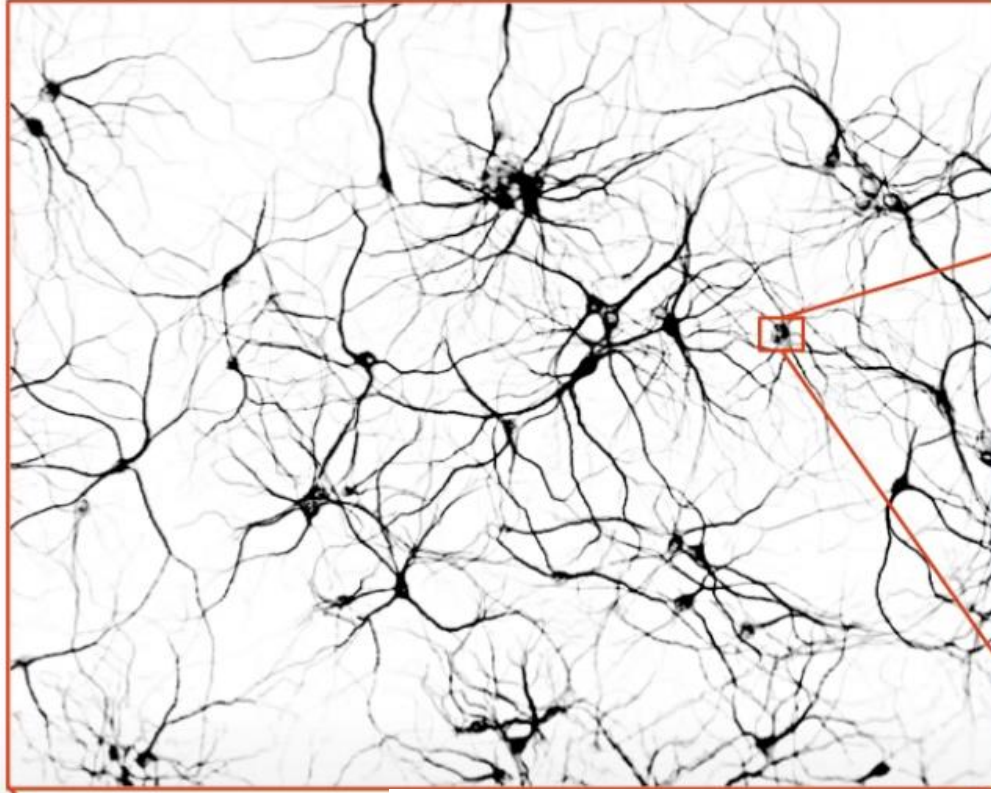


Introduction to Artificial Neural Networks (ANNs)

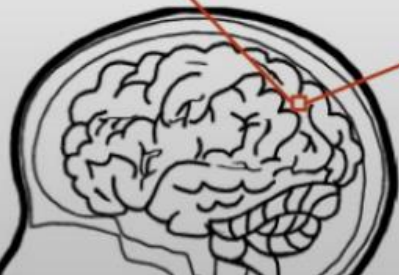
The brain

- The brain is the human's learning mechanism
- 20th century research found that brain can be described a network of neurons
- Neurons are nerve cells and sensitive to input
- Estimates suggest there are 10B neurons and 10T synapses!





- Neurons receive electro-chemical signals and become 'excited'
- If an excitation **threshold** is exceeded, the neuron fires (electric signal)
- Electrical pulse **propagated** along axon to synaptic junction
- When synapse receives **input**, releases neurotransmitters, which enhance signals over time – **training**

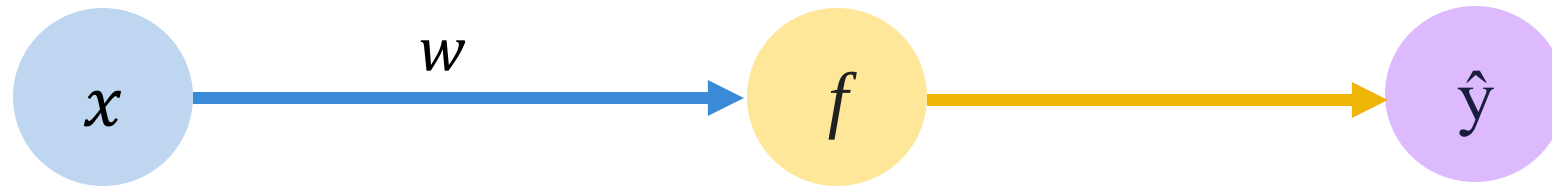


The Perceptron

The Perceptron

- Artificial neuron (perceptron), in much the same way as human brain, has:
 - a number of **inputs** (x);
 - synapses that are **weighted** (w) and variable
 - a single **output** (axon) (y)
 - an excitation **threshold**, which once exceeded leads to the neuron firing – transfer (**activation**) **function**

Simplest Perceptron (one input)



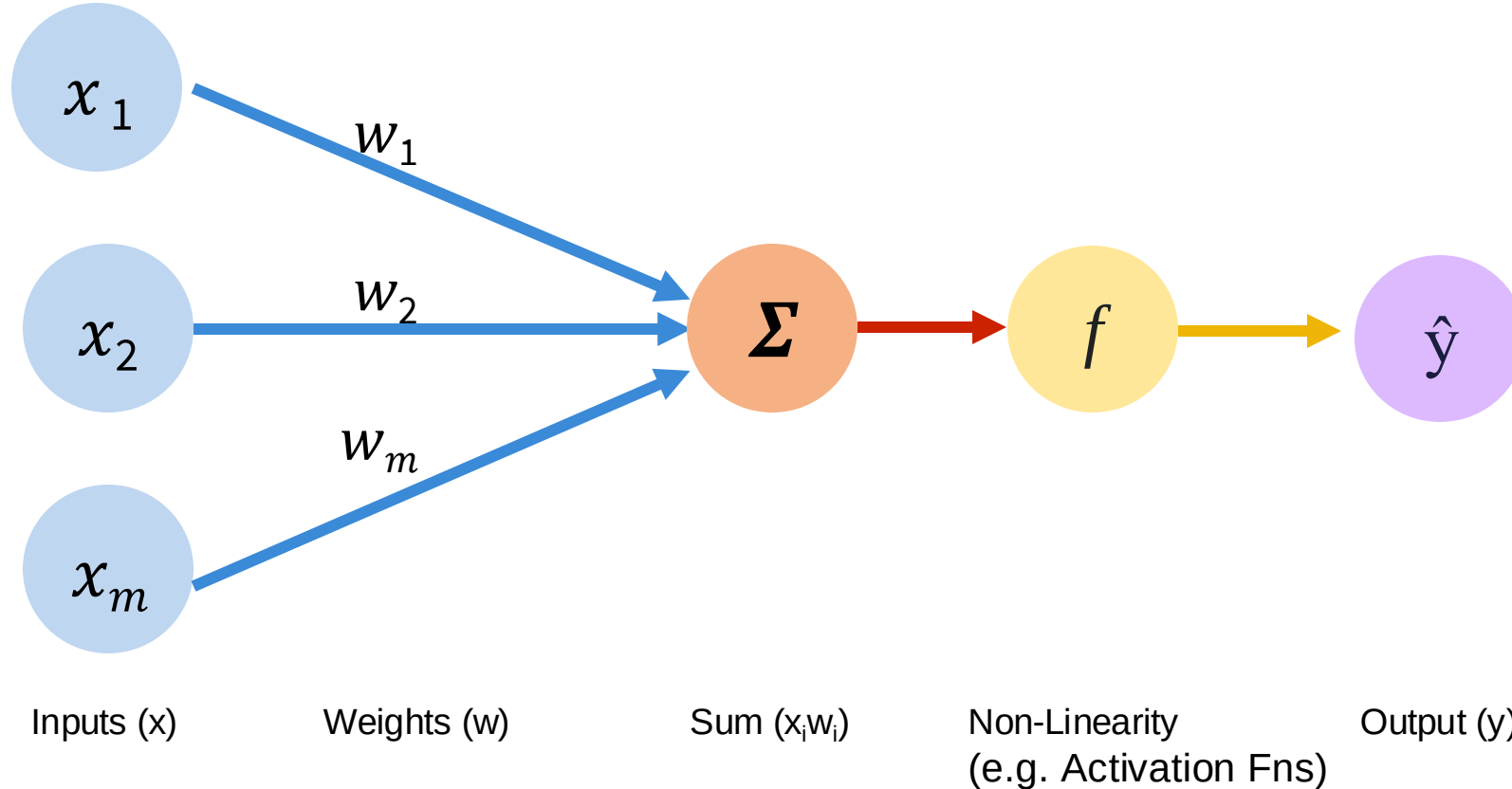
Input (x)

Weight (w)

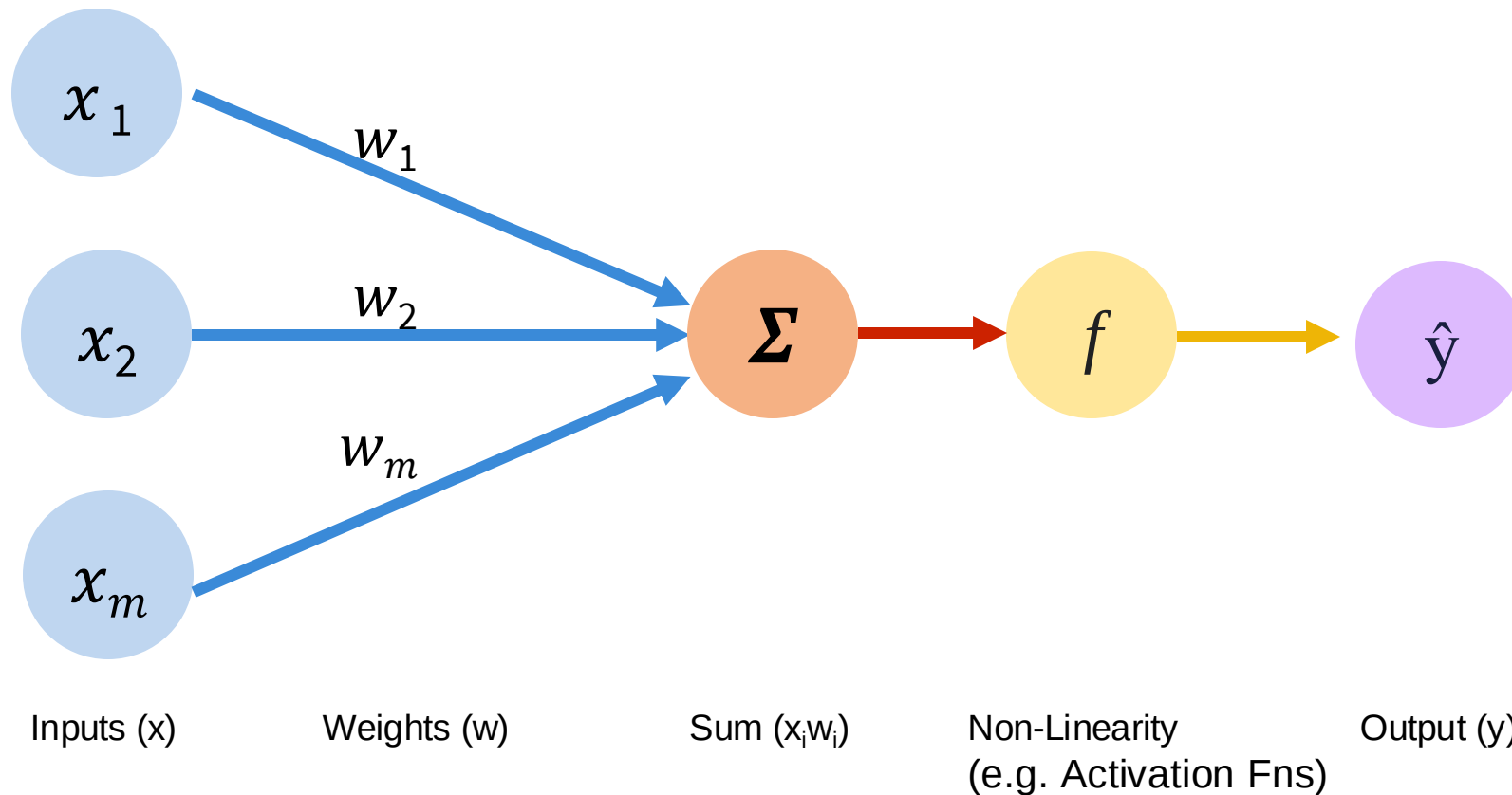
Non-Linearity
(e.g. Activation Fns)

Output (y)

The Perceptron (multiple inputs)



The Perceptron (multiple inputs)



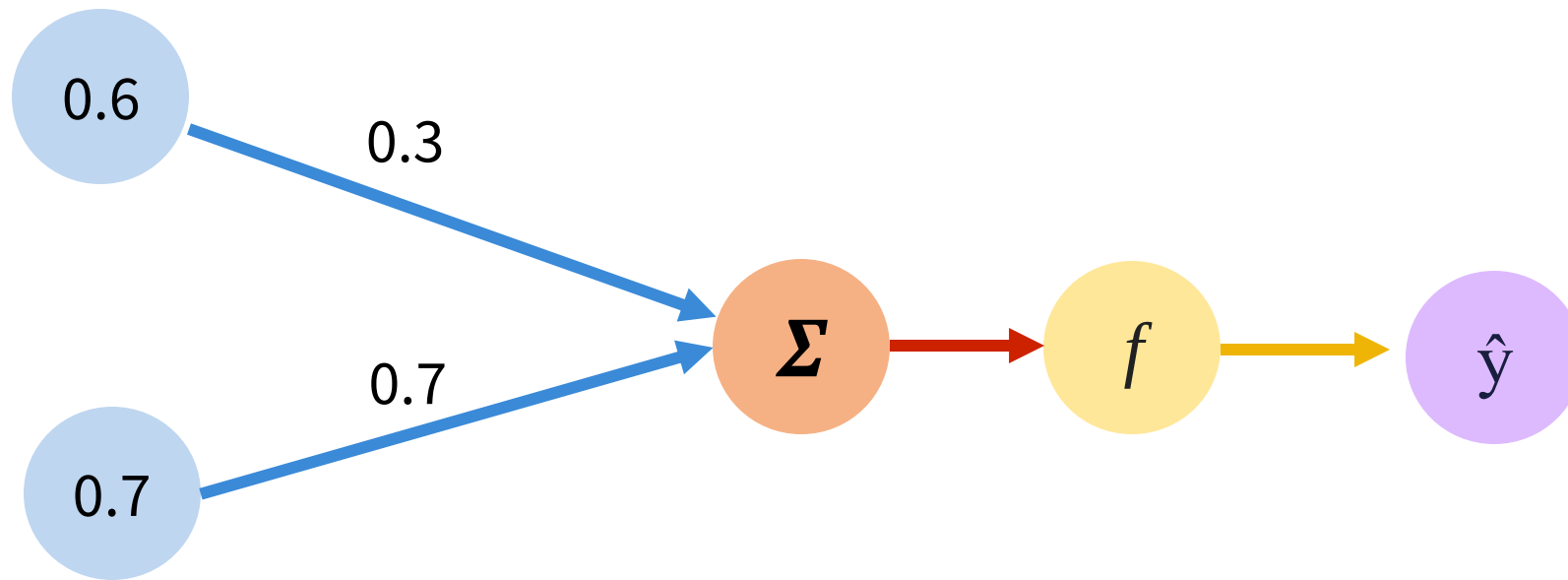
Output

Linear combination of inputs

$$\hat{y} = g \left(\sum_{i=1}^m x_i w_i \right)$$

Non-Linear activation function

The Perceptron (Concrete)



Inputs (x)

Weights (w)

Sum ($x_i w_i$)

Non-Linearity
(e.g. Activation Fns)

Output (y)

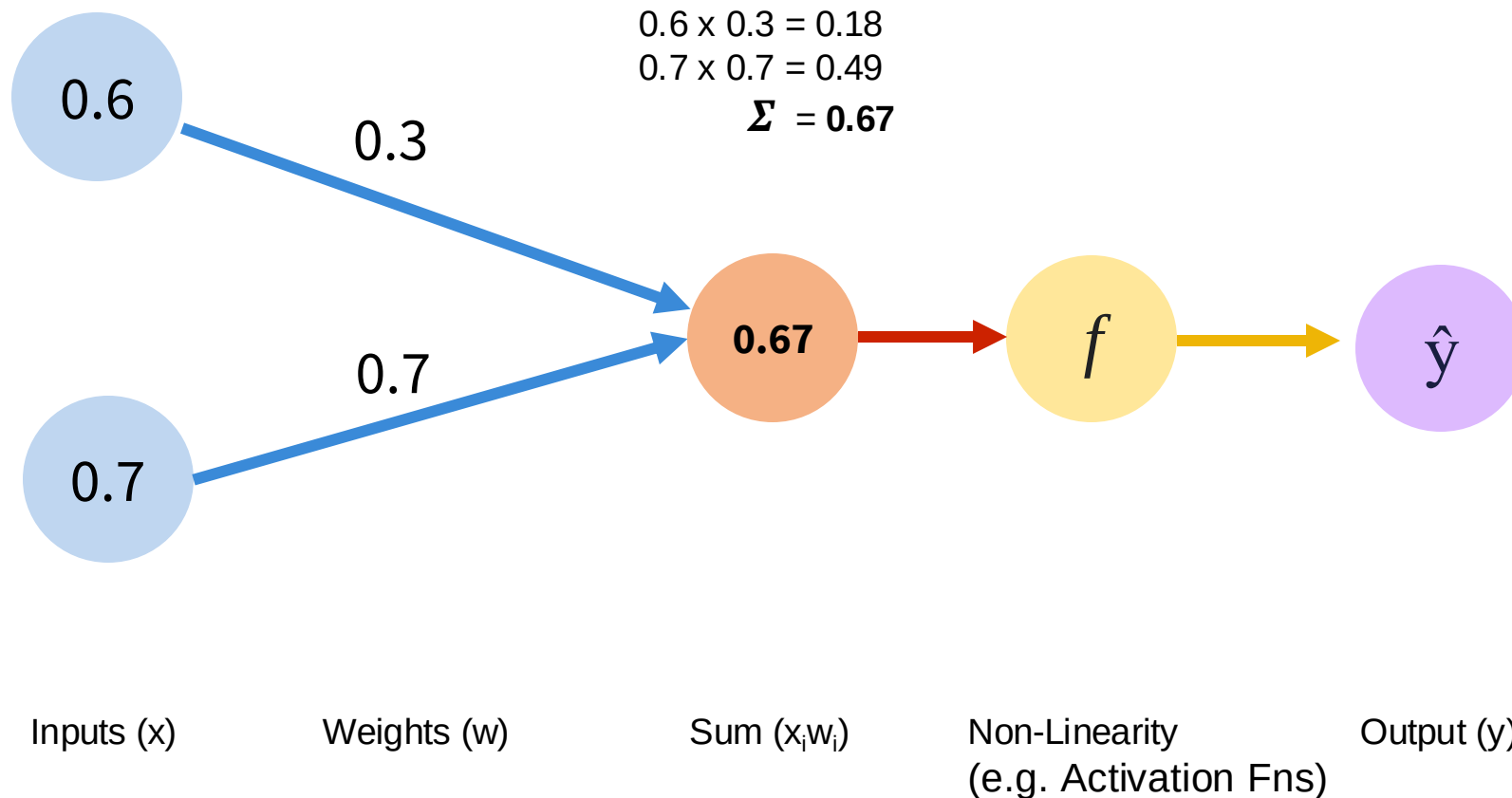
Output

Linear combination of inputs

$$\hat{y} = g \left(\sum_{i=1}^m x_i w_i \right)$$

Non-Linear activation function

The Perceptron (Concrete)



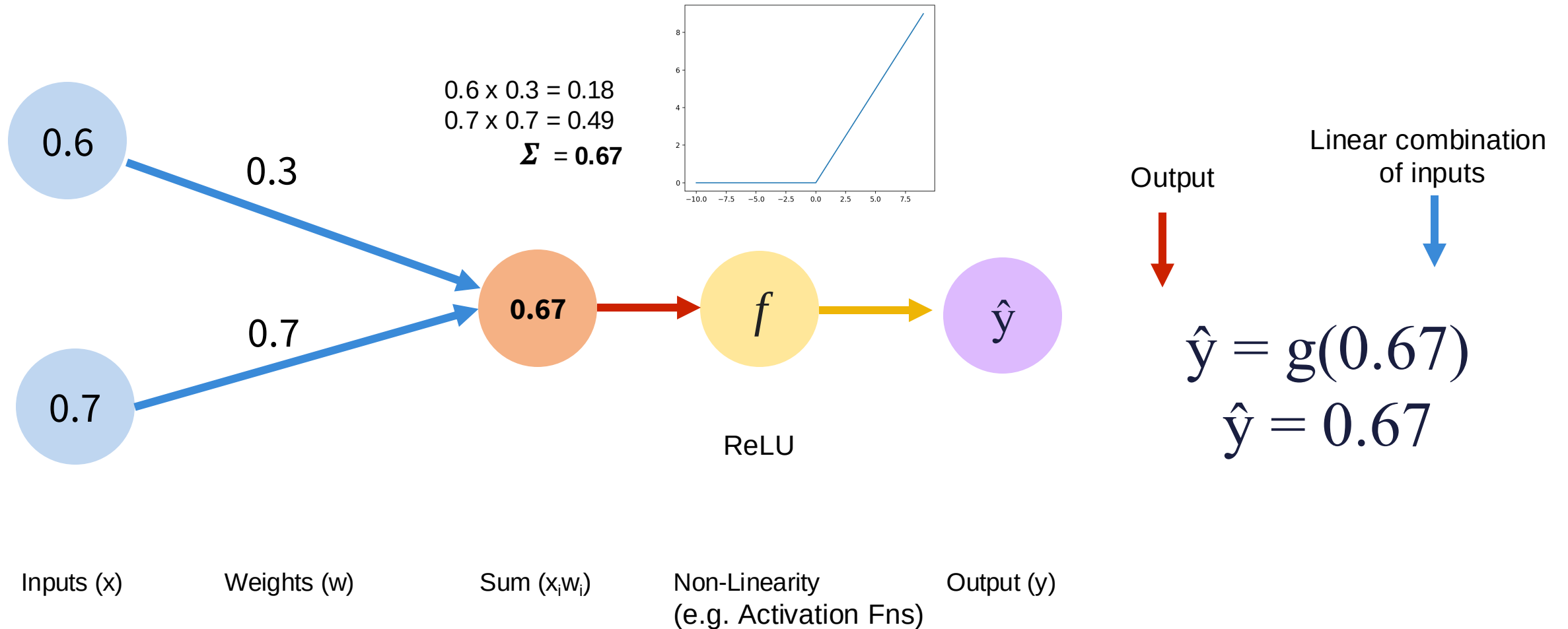
Output

Linear combination of inputs

$$\hat{y} = g \left(\sum_{i=1}^m x_i w_i \right)$$

Non-Linear activation function

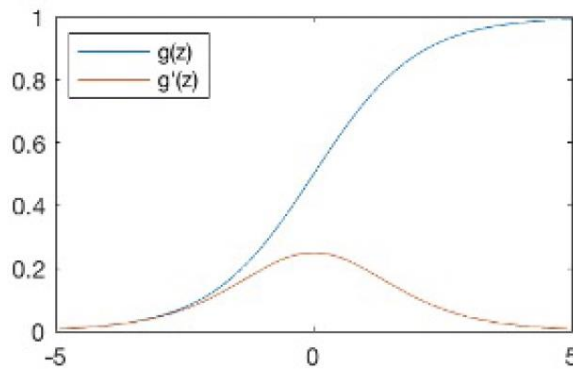
The Perceptron (Concrete) - ReLU



Activation Functions

Common Activation Functions

Sigmoid Function

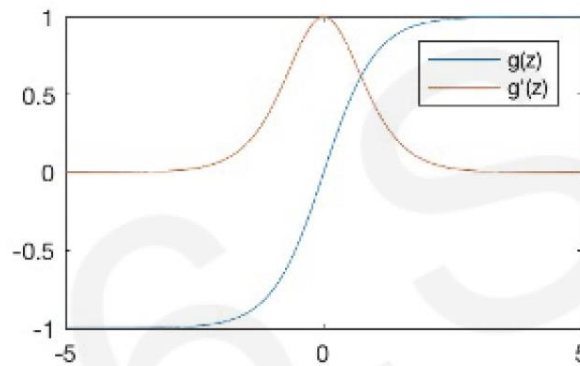


$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

 `tf.math.sigmoid(z)`

Hyperbolic Tangent

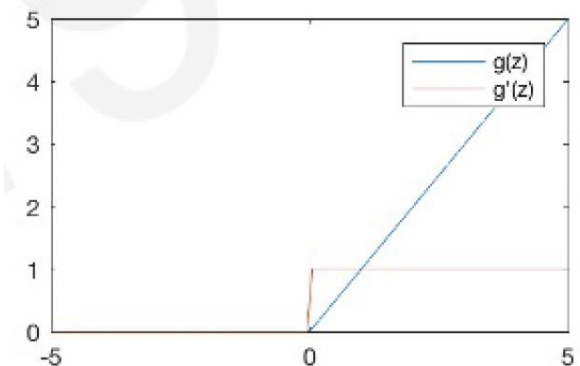


$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

 `tf.math.tanh(z)`

Rectified Linear Unit (ReLU)

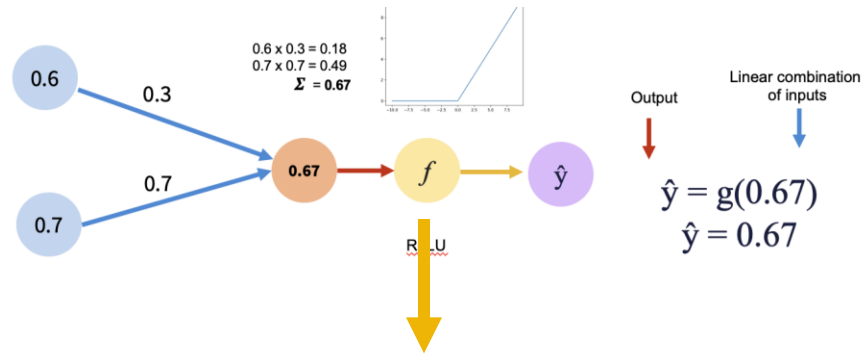


$$g(z) = \max(0, z)$$

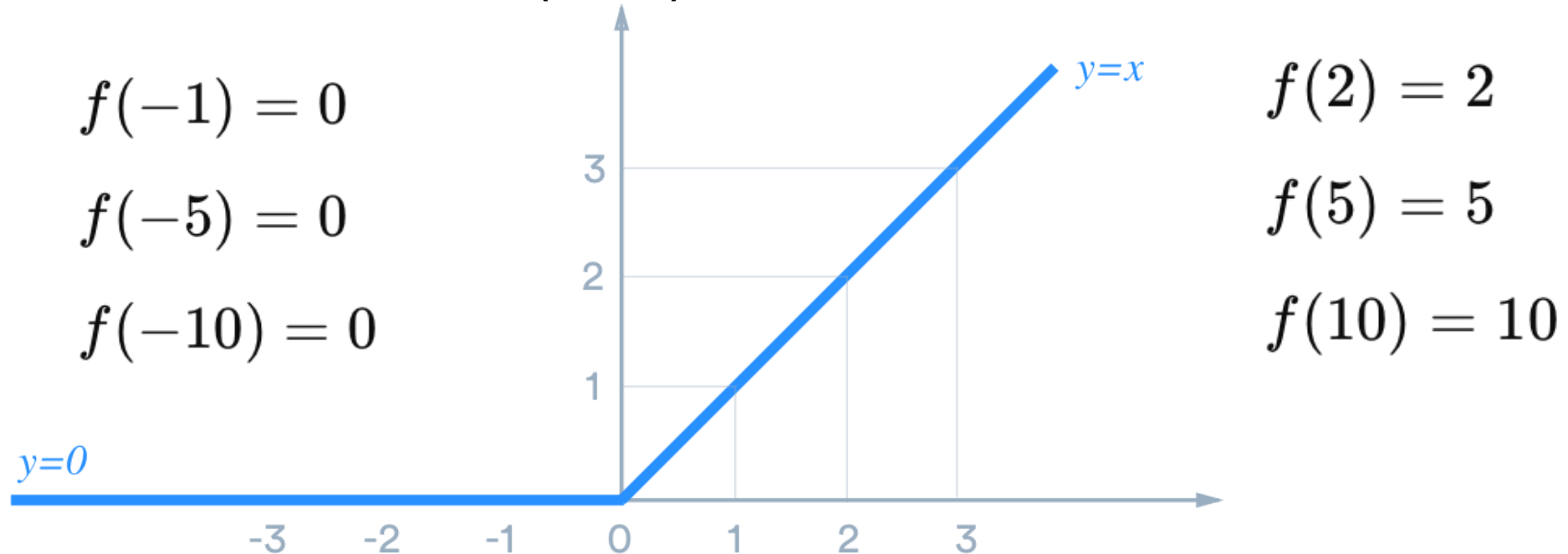
$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

 `tf.nn.relu(z)`

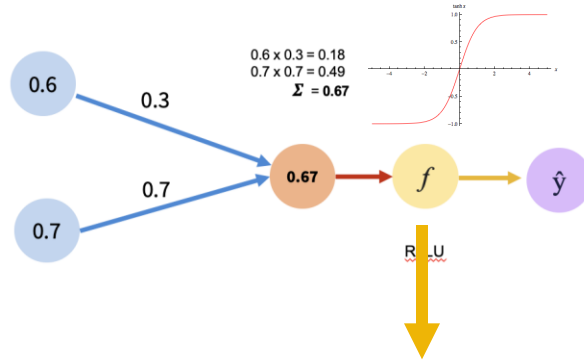
Common Activation Functions - ReLU



$$\text{ReLU}(0.67) = \mathbf{0.67}$$



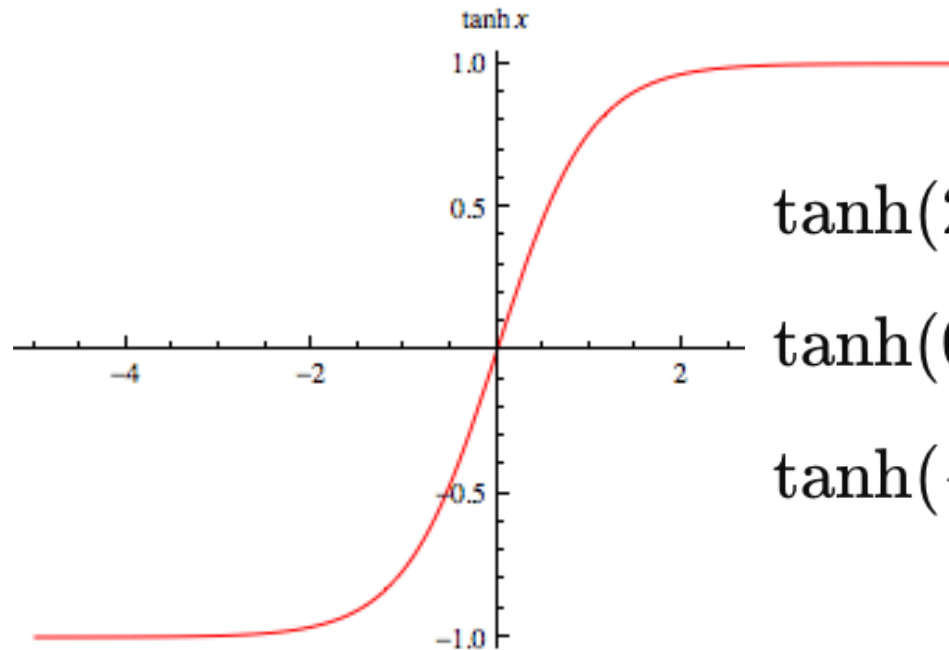
Common Activation Functions - tanh



tanh will activate with a value in the range $[-1, 1]$.

ReLU simply turns off negative values but tanh squashes the input into smaller range symmetrically around zero.

$$\text{Tanh}(0.67) = \mathbf{0.58497..}$$



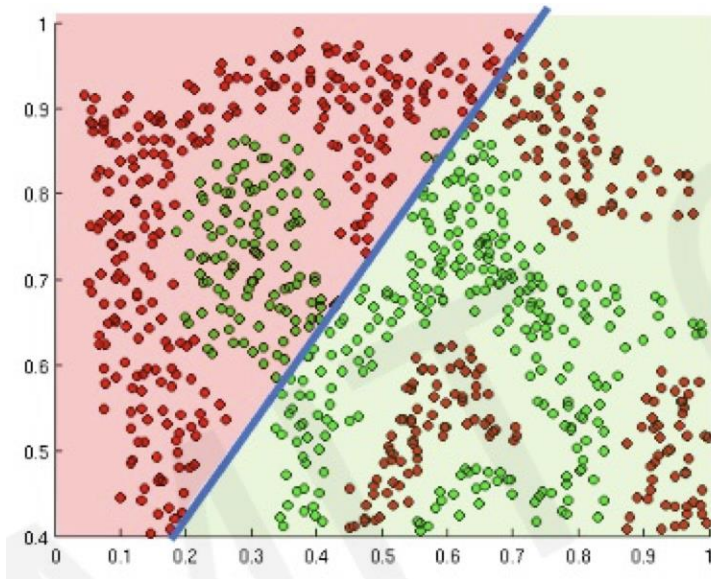
$\tanh(2)$ will be positive and close to 1

$\tanh(0)$ will be 0

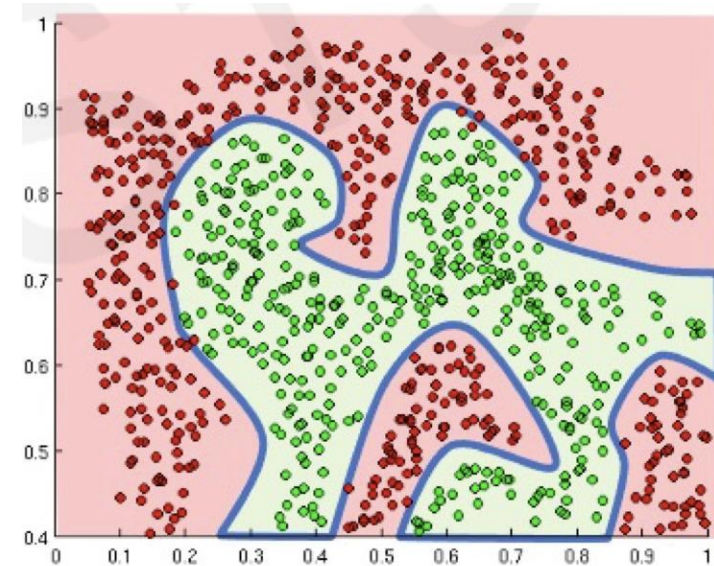
$\tanh(-2)$ will be negative and close to -1

Importance of Activation Functions

- The purpose of activation functions is to **introduce non-linearities** into the network



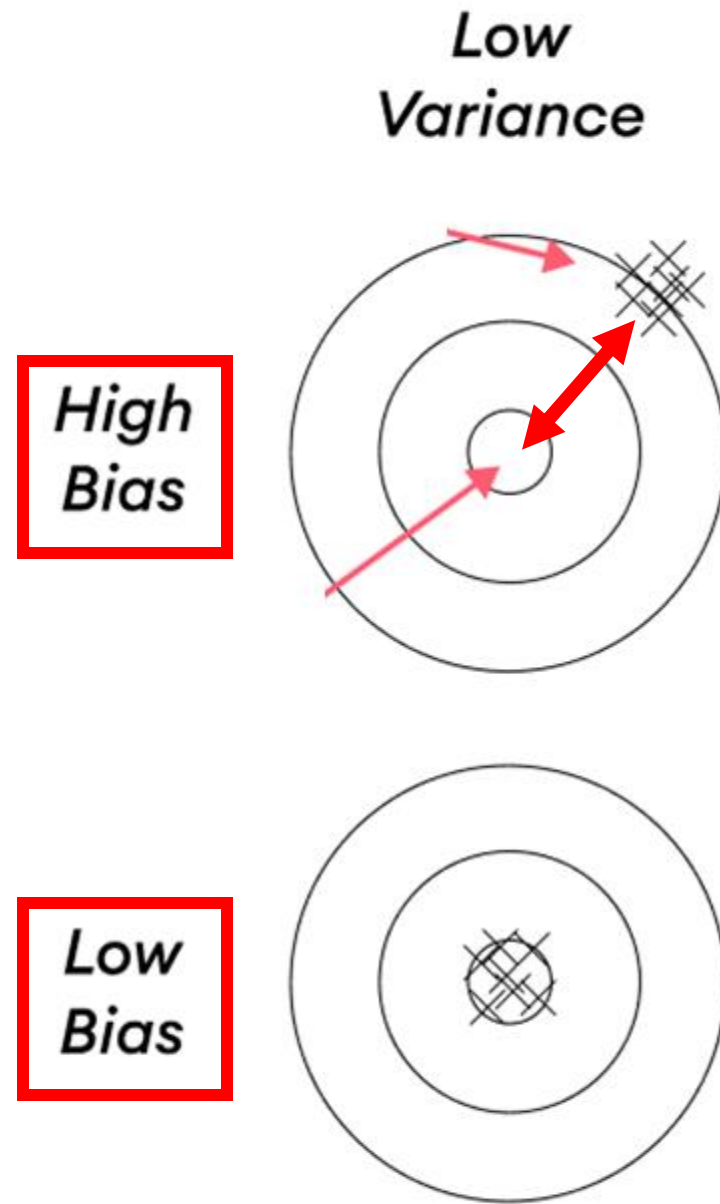
Linear activation functions produce linear decisions no matter the network size



Non-linearities allow us to approximate arbitrarily complex functions

Bias

Bias

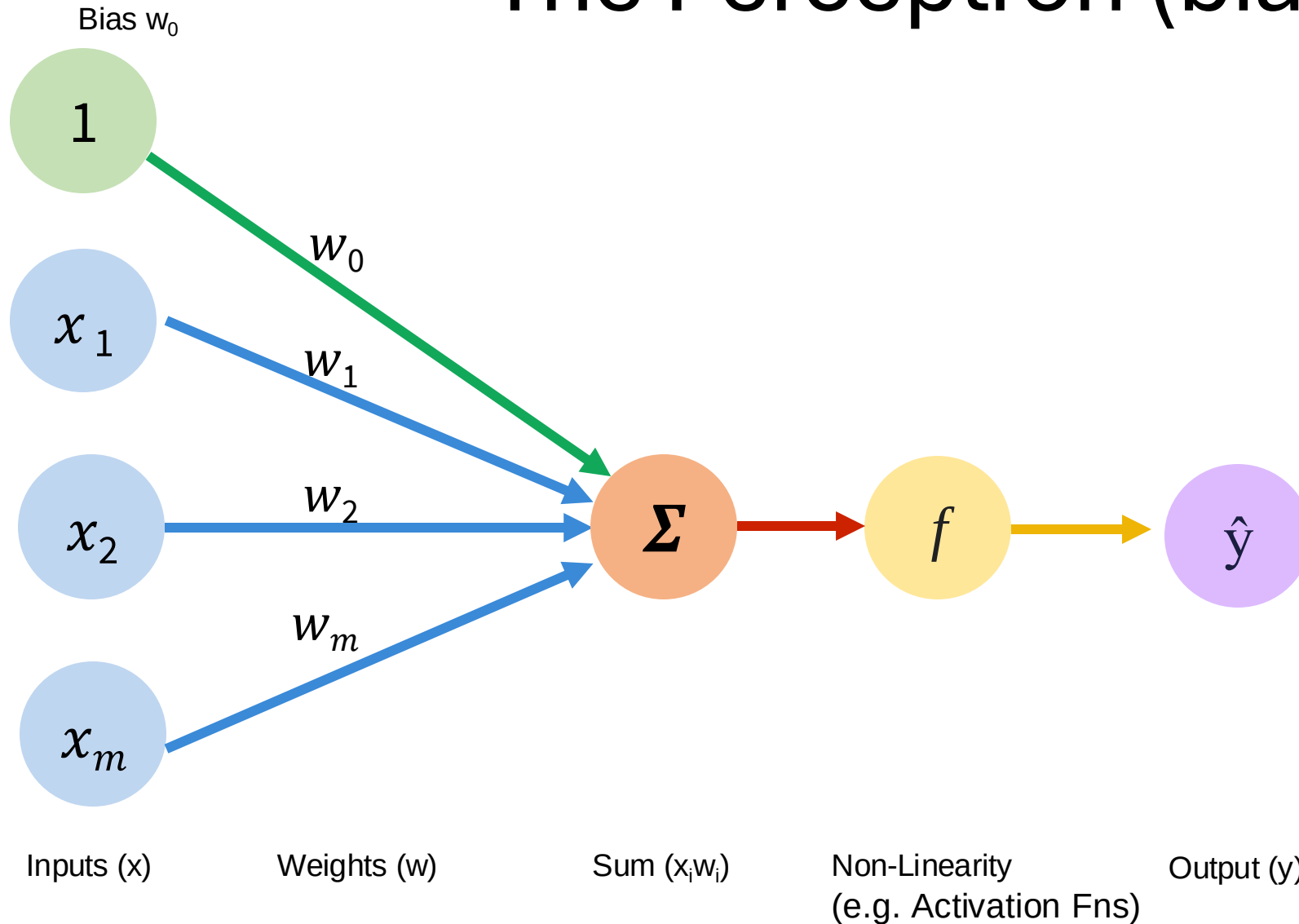


Bias

- Bias can be seen as a weighted connection, whose value is typically 1
- It is added to the product of weight and input
- When no bias is used a fixed threshold θ is set
- Threshold of 0 commonly used when bias added
- $a = \begin{cases} +1 & \text{if } n \geq 0 \\ 0 & \text{if } n < 0 \end{cases}$

$$n = b + \sum_i x_i w_i$$

The Perceptron (bias)

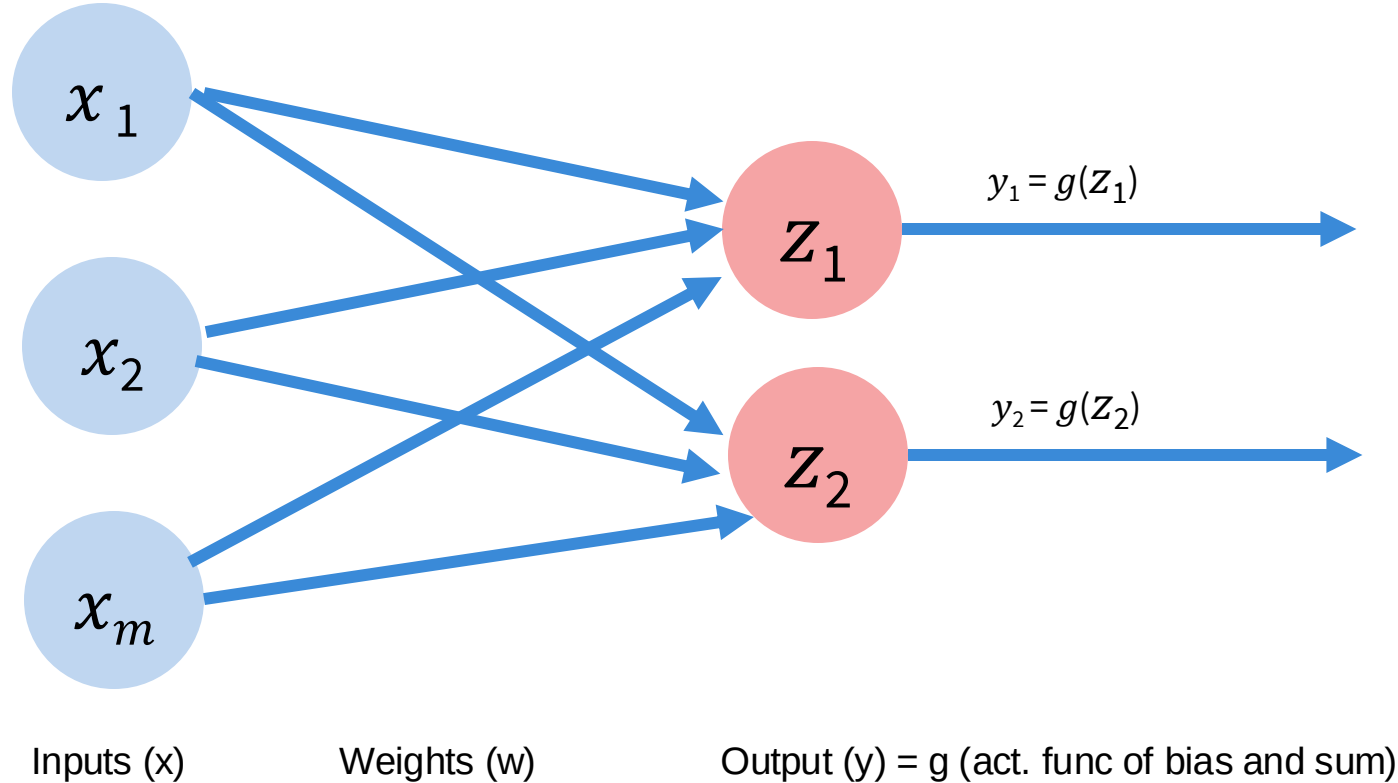


The equation for the output of the perceptron is shown, with color-coded arrows pointing to specific parts of the formula:

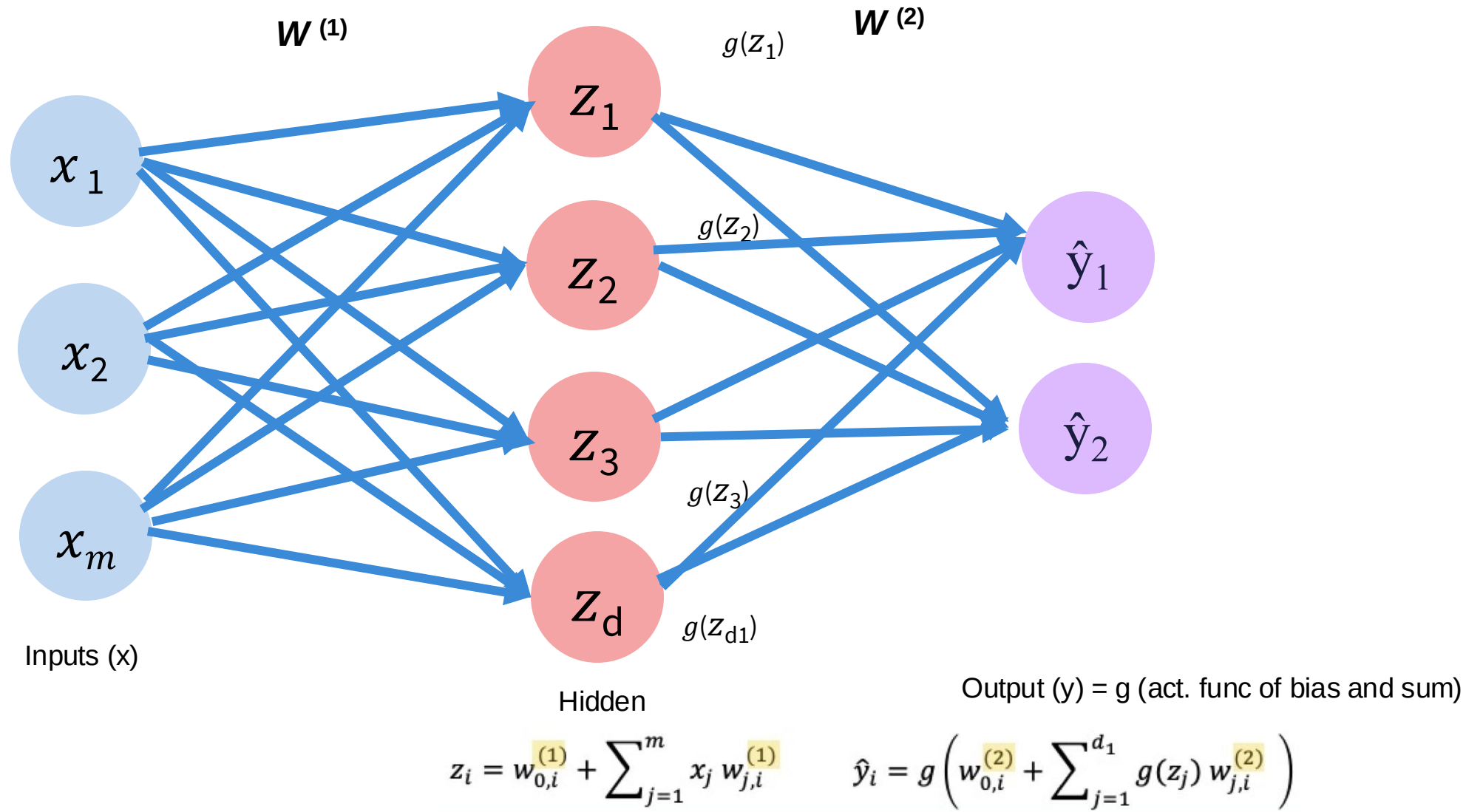
$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

- A purple arrow points from the word "Output" to $\hat{y}$.
- A red arrow points from the text "Linear combination of inputs" to the summation term $\sum_{i=1}^m x_i w_i$.
- A green arrow points from the word "Bias" to the w_0 term.
- A yellow arrow points from the text "Non-linear activation function" to the g function.

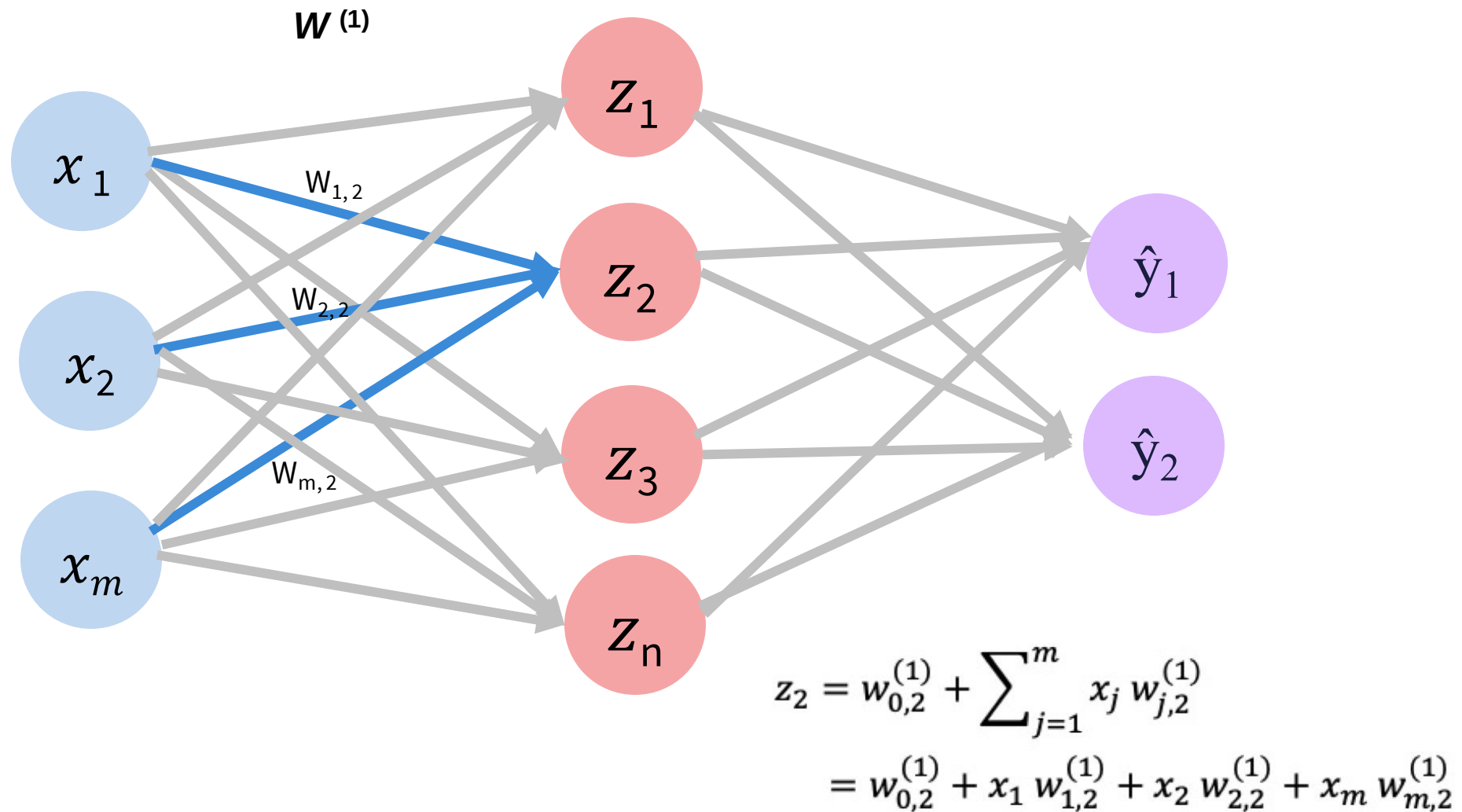
The Perceptron (multiple outputs)



Single Layer Neural Network

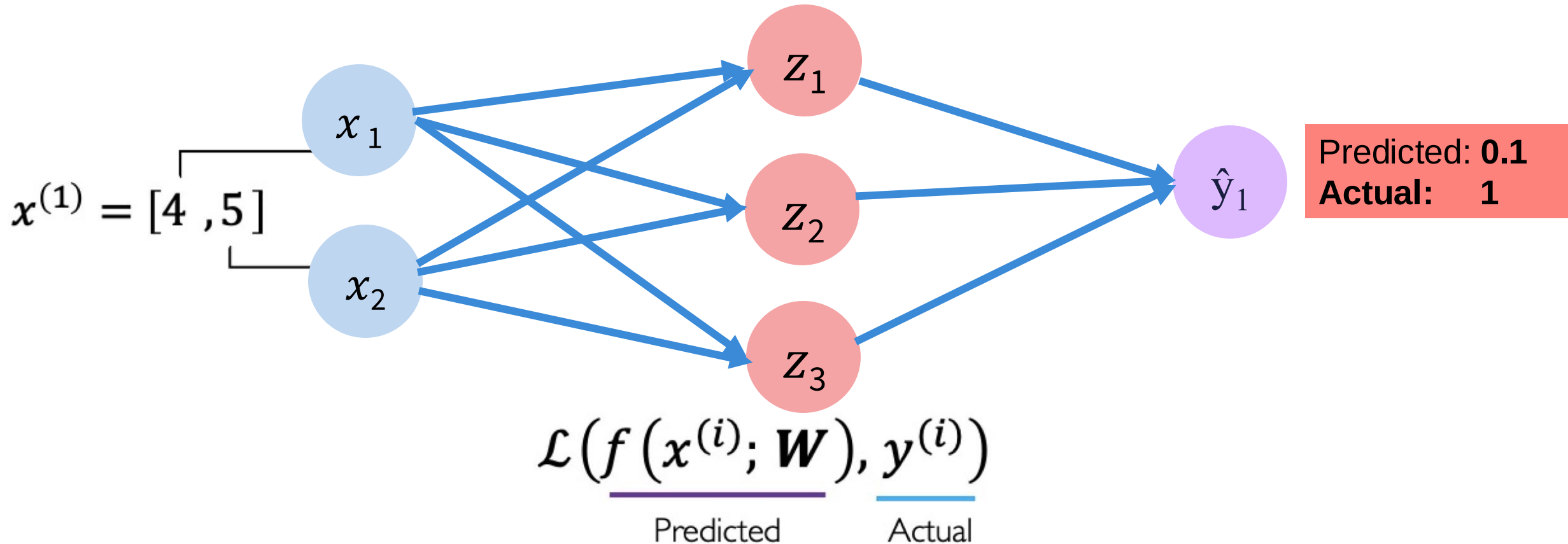


Single Layer Neural Network



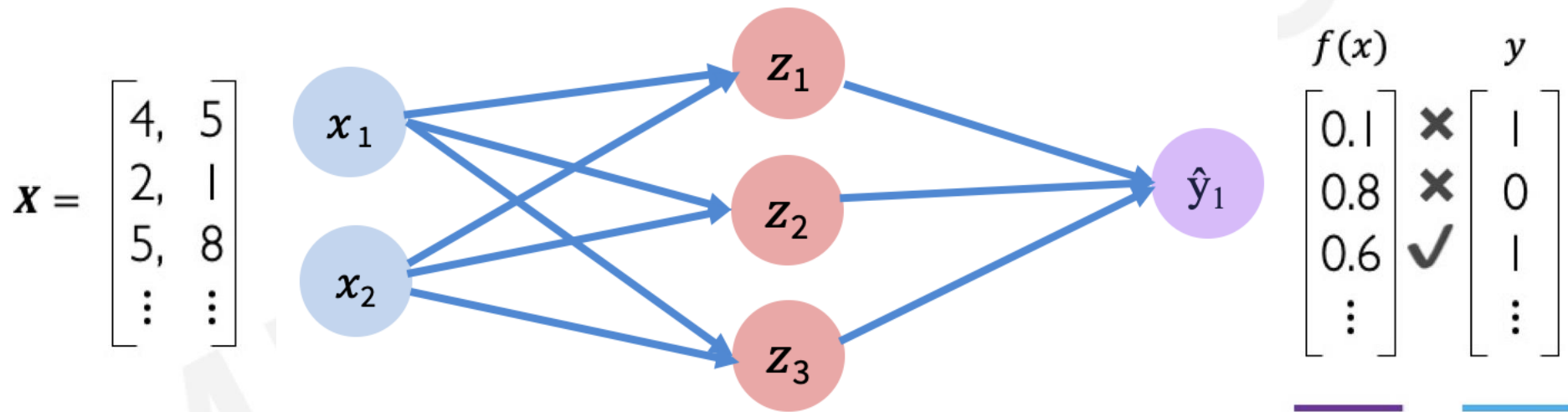
Quantifying Loss

- The ***loss of our network*** measures the **cost incurred from incorrect predictions**.



Empirical Loss

- The **empirical loss** measures the **total loss over our entire dataset**.



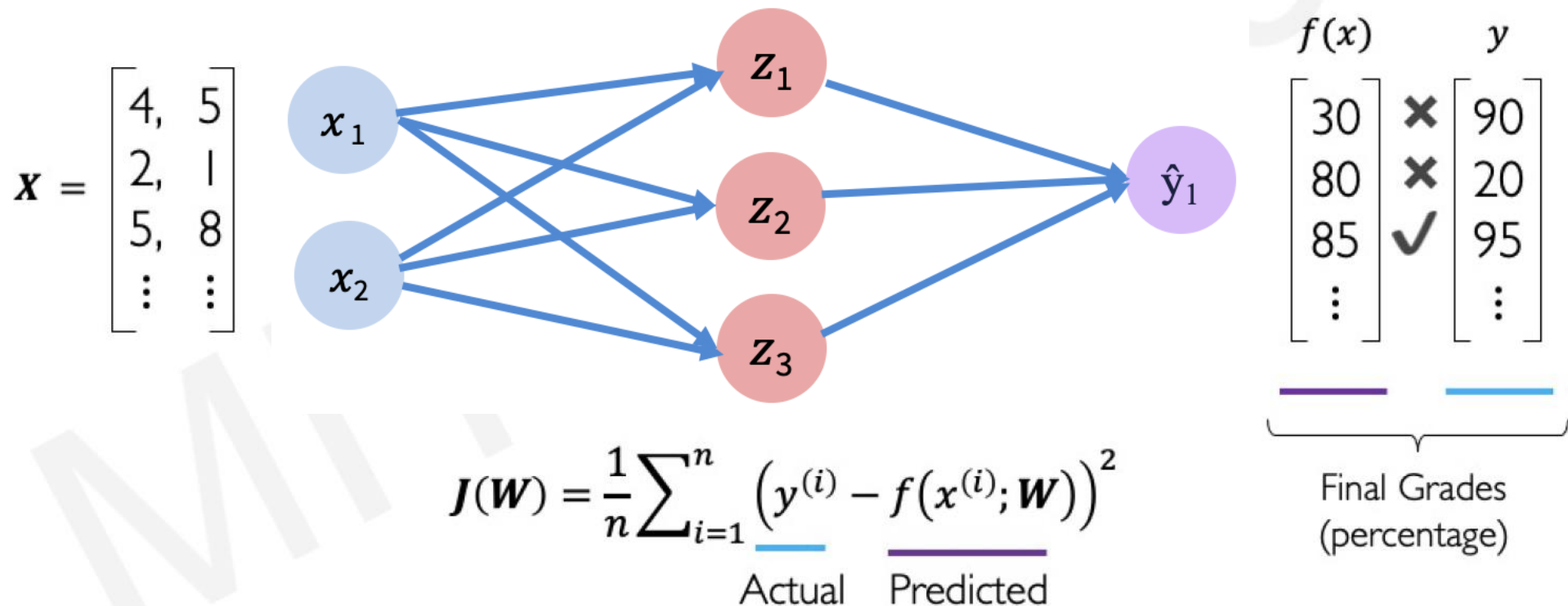
Also known as:

- Objective function
- Cost function
- Empirical Risk

$$J(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Mean Squared Error Loss

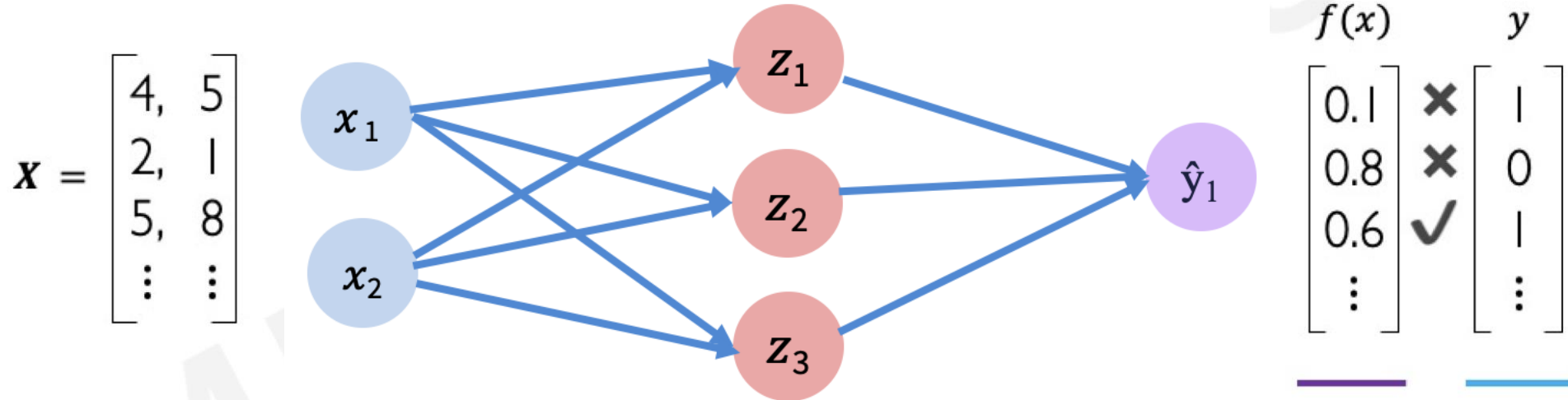
- **Mean squared error loss** can be used with regression models that output continuous real numbers.



```
loss = tf.reduce_mean( tf.square(tf.subtract(y, predicted)) )  
loss = tf.keras.losses.MSE( y, predicted )
```

Binary Cross Entropy Loss

- **Cross entropy loss** can be used with models that output a **probability** between 0 and 1.



$$J(\mathbf{W}) = -\frac{1}{n} \sum_{i=1}^n \underbrace{y^{(i)}}_{\text{Actual}} \log \left(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}} \right) + (1 - \underbrace{y^{(i)}}_{\text{Actual}}) \log \left(1 - \underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}} \right)$$



```
loss = tf.reduce_mean( tf.nn.softmax_cross_entropy_with_logits(y, predicted) )
```

Training a Neural Network

Loss Optimization

- We want to find the network weights that **achieve the lowest loss**

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

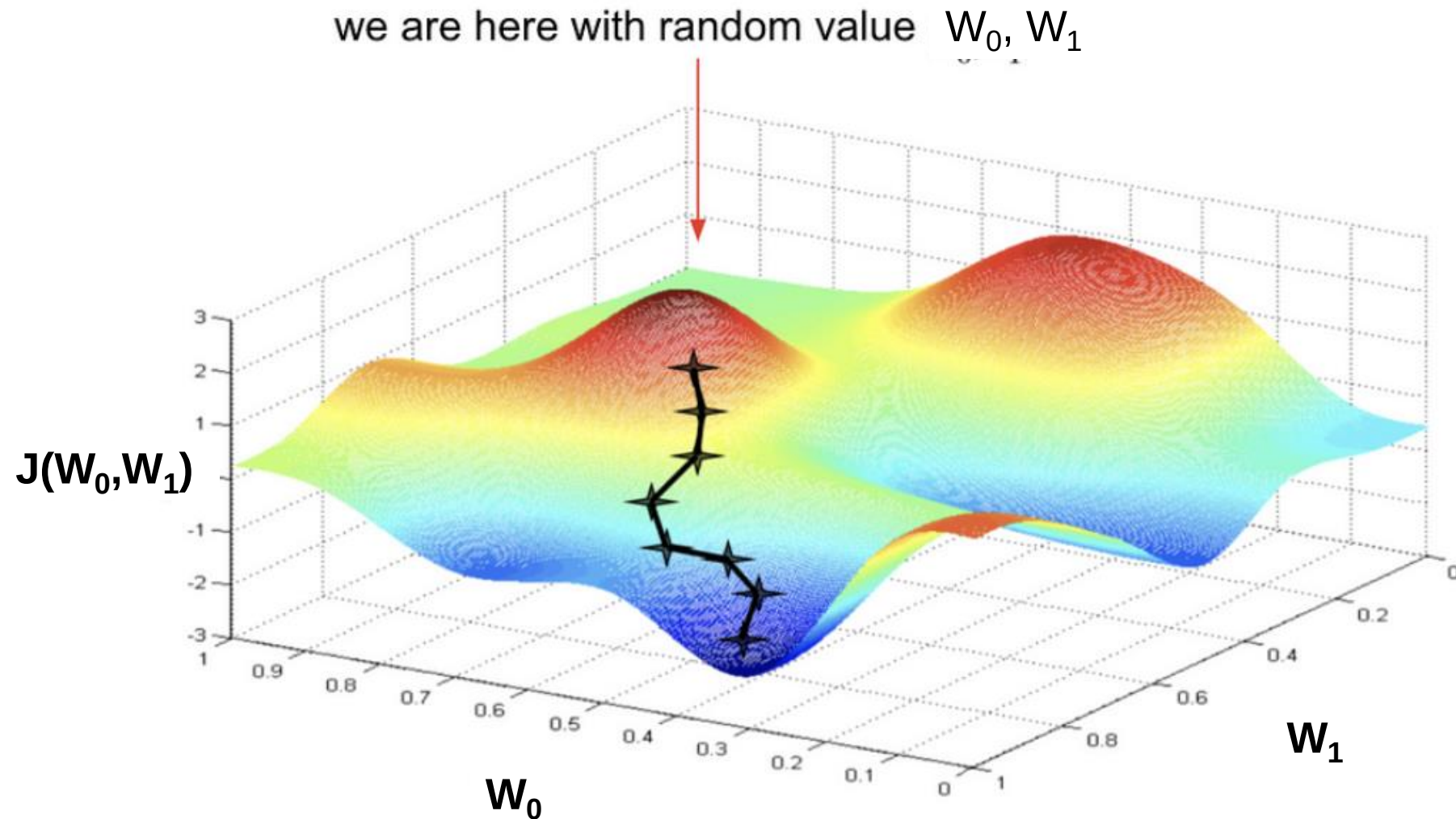
$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$



Remember:

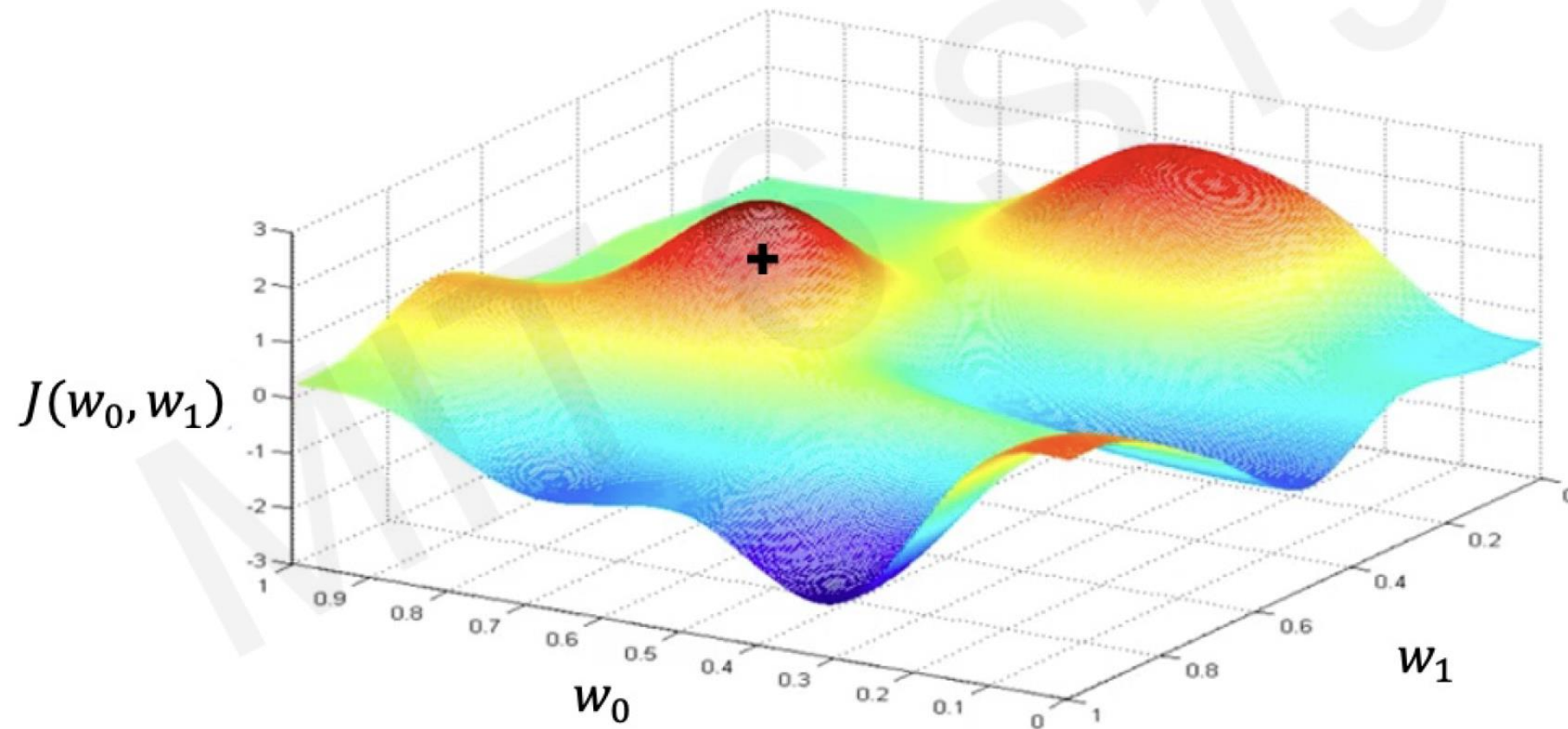
$$\mathbf{W} = \{\mathbf{W}^{(0)}, \mathbf{W}^{(1)}, \dots\}$$

Loss Optimization via gradient descent

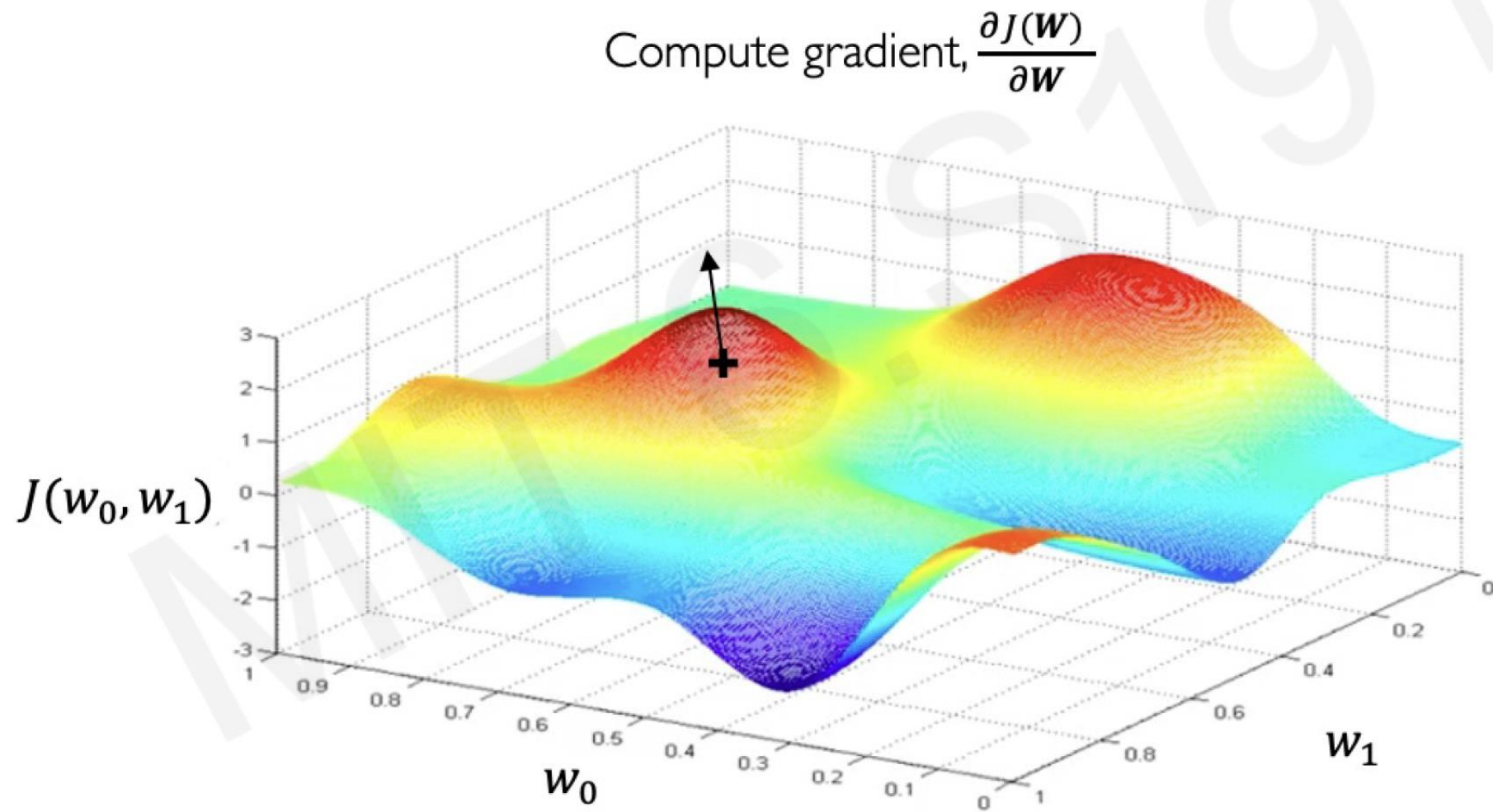


Loss Optimization

Randomly pick an initial (w_0, w_1)

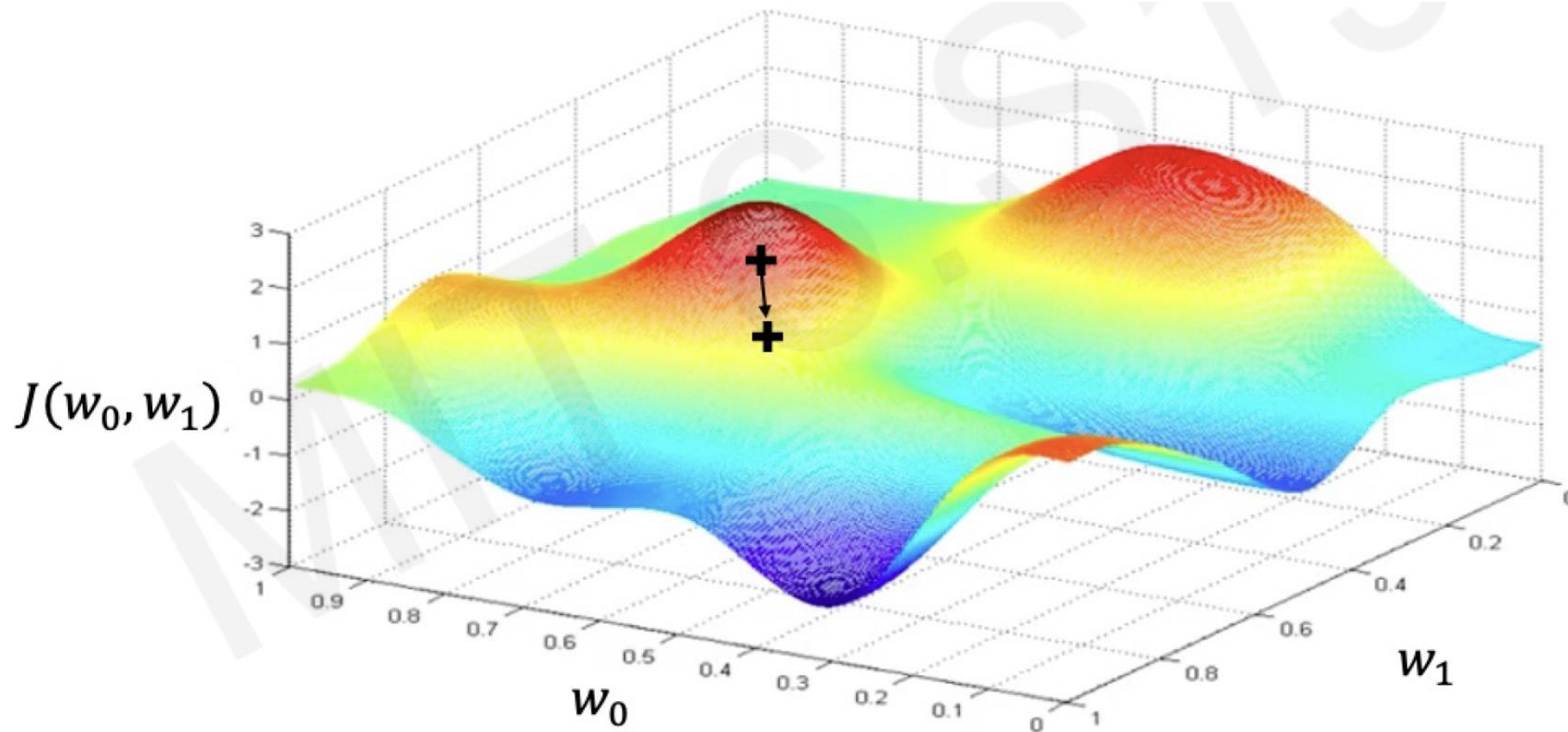


Loss Optimization



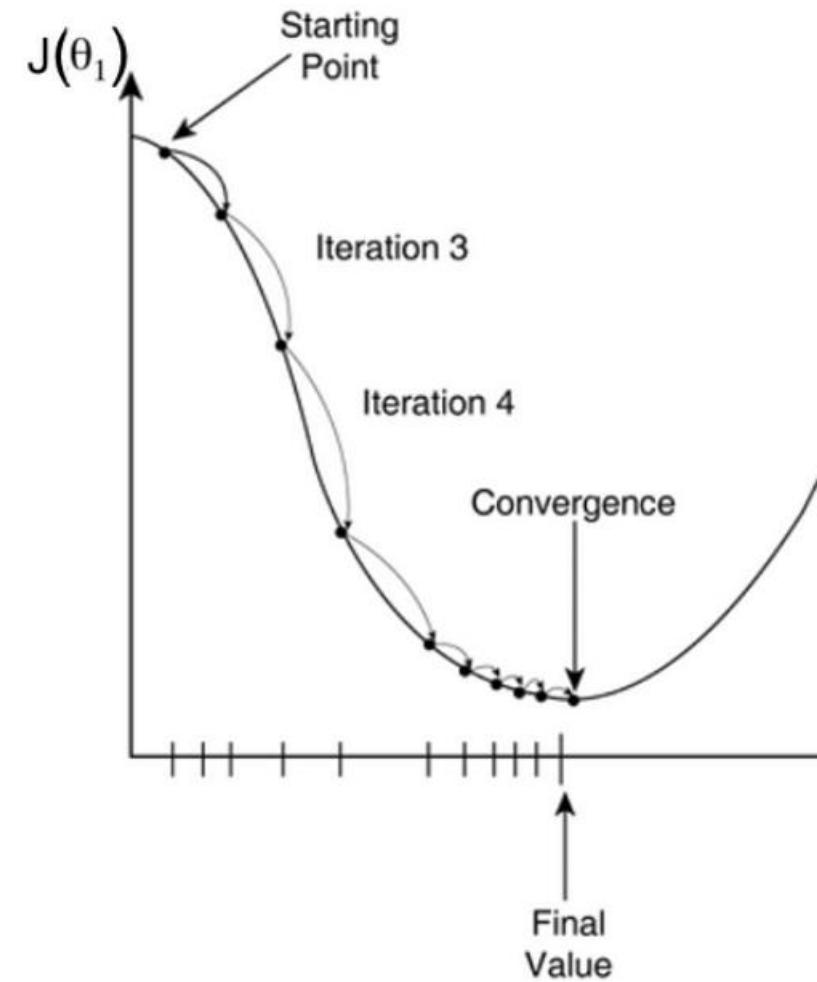
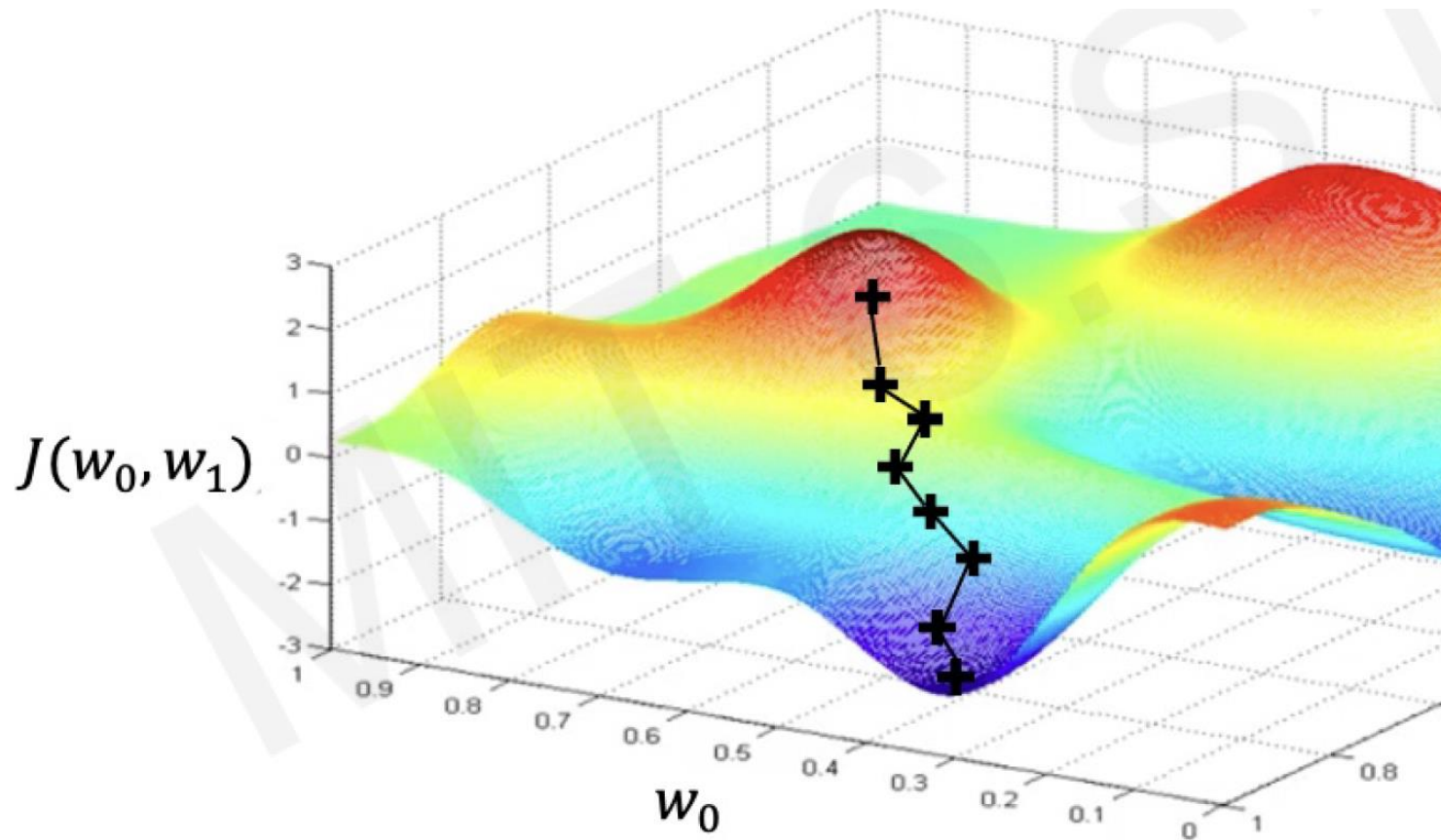
Loss Optimization

Take small step in opposite direction of gradient



Gradient Descent

Repeat until convergence



Gradient Descent

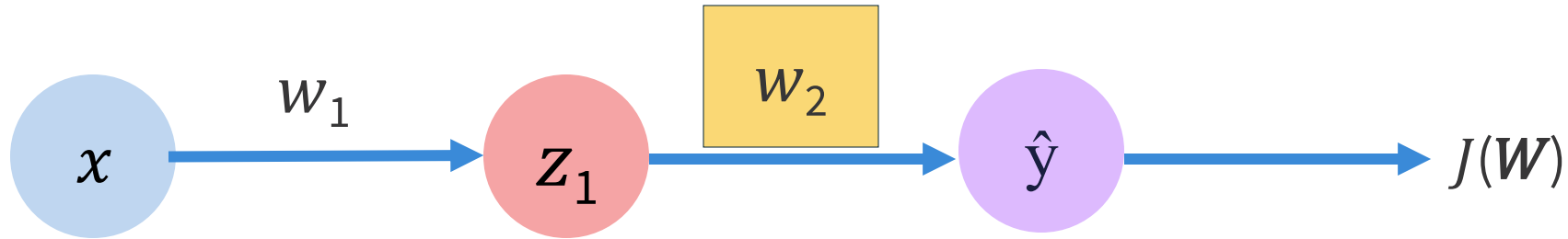
Algorithm

1. Initialise weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient $\frac{\partial J(W)}{\partial W}$
4. Update weights $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

The idea is to **reduce loss** through **training**

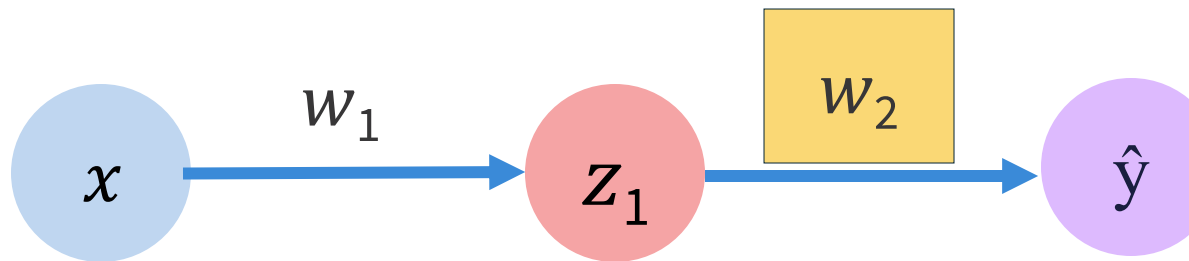
```
Epoch 1/20
2024-05-13 13:28:55.223274: I tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry.cc:117
219/219 [=====] - 3s 5ms/step - loss: 0.1955 - root_mean_squared_error: 0.4422
Epoch 2/20
219/219 [=====] - 1s 4ms/step - loss: 0.0552 - root_mean_squared_error: 0.2350
Epoch 3/20
219/219 [=====] - 1s 4ms/step - loss: 0.0309 - root_mean_squared_error: 0.1758
Epoch 4/20
219/219 [=====] - 1s 4ms/step - loss: 0.0208 - root_mean_squared_error: 0.1442
Epoch 5/20
219/219 [=====] - 1s 4ms/step - loss: 0.0157 - root_mean_squared_error: 0.1253
Epoch 6/20
219/219 [=====] - 1s 4ms/step - loss: 0.0129 - root_mean_squared_error: 0.1137
Epoch 7/20
219/219 [=====] - 1s 4ms/step - loss: 0.0113 - root_mean_squared_error: 0.1063
Epoch 8/20
219/219 [=====] - 1s 4ms/step - loss: 0.0101 - root_mean_squared_error: 0.1007
```

Computing Gradients: Backpropagation



How does a small in change in one weight (ex, w_2) affect the final loss $J(W)$?

Computing Gradients: Backpropagation



$$\frac{\partial J(W)}{\partial w_2} =$$

Let's use the **chain rule** (calculus)

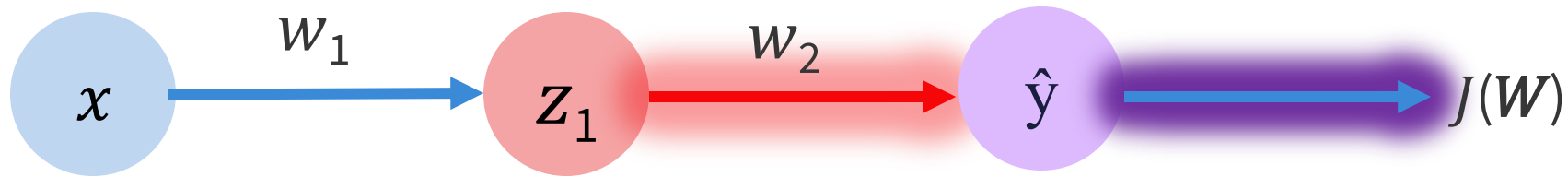
$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$$

$$\frac{dy}{dx} = \text{derivative of } y \text{ with respect to } x$$

$$\frac{dy}{du} = \text{derivative of } y \text{ with respect to } u$$

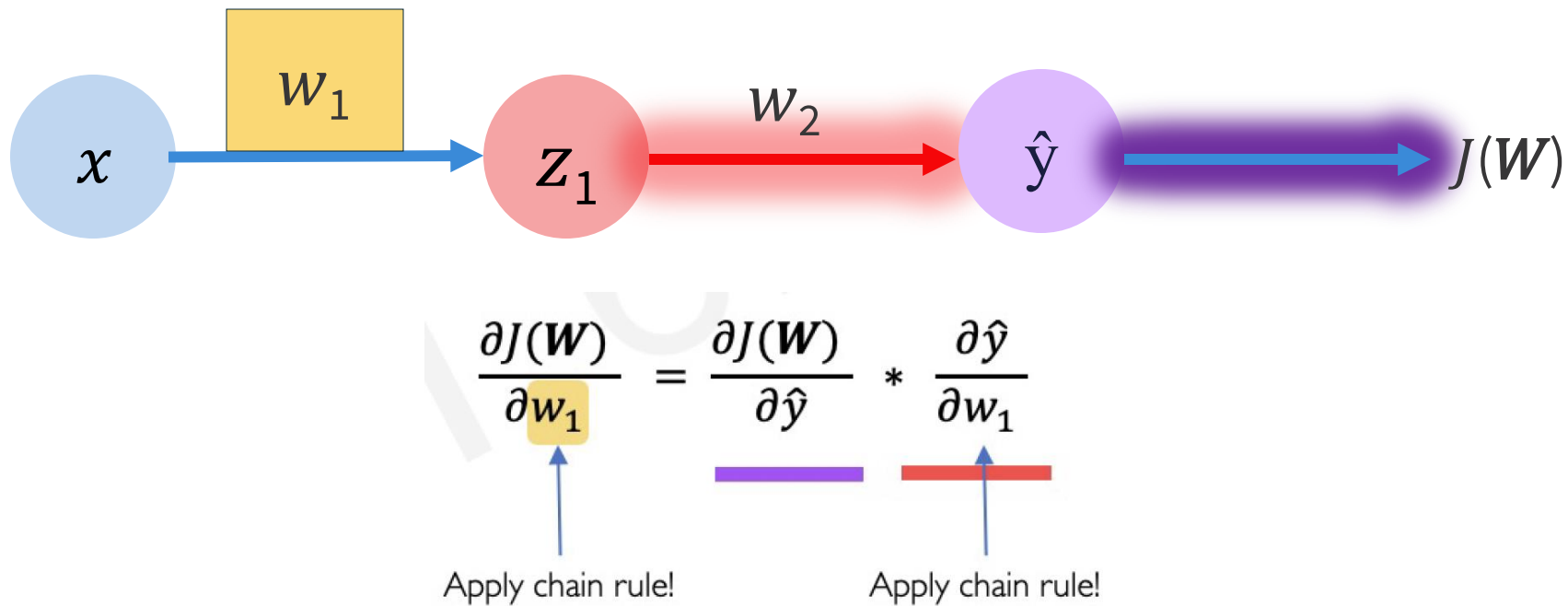
$$\frac{du}{dx} = \text{derivative of } u \text{ with respect to } x$$

Computing Gradients: Backpropagation

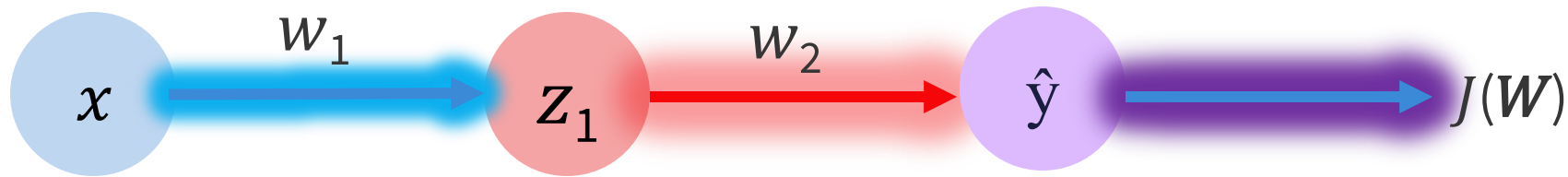


$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_2}$$

Computing Gradients: Backpropagation



Computing Gradients: Backpropagation

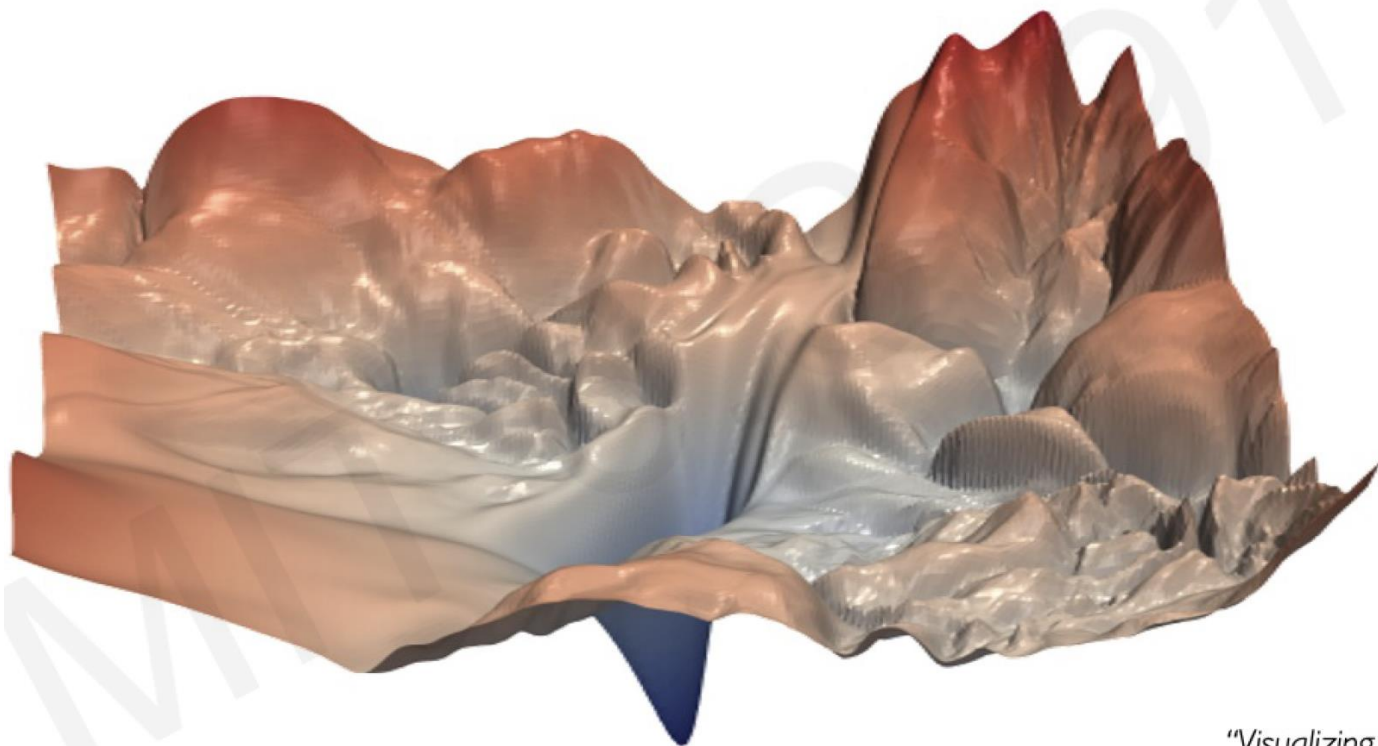


$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

Repeat this for every weight in the network using gradients from later layers

Neural Networks in Practice: Optimization

Training Neural Networks is Difficult

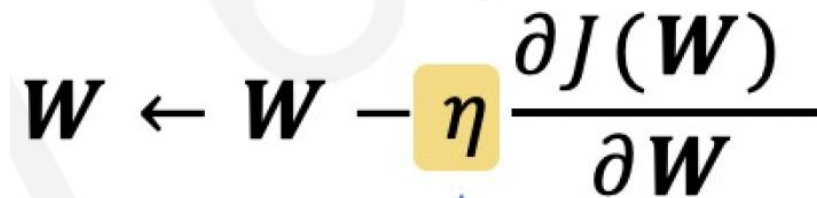


"Visualizing the loss landscape of neural nets". Dec 2017.

Loss functions can be difficult to Optimize

Remember:

Optimization through gradient descent


$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$$

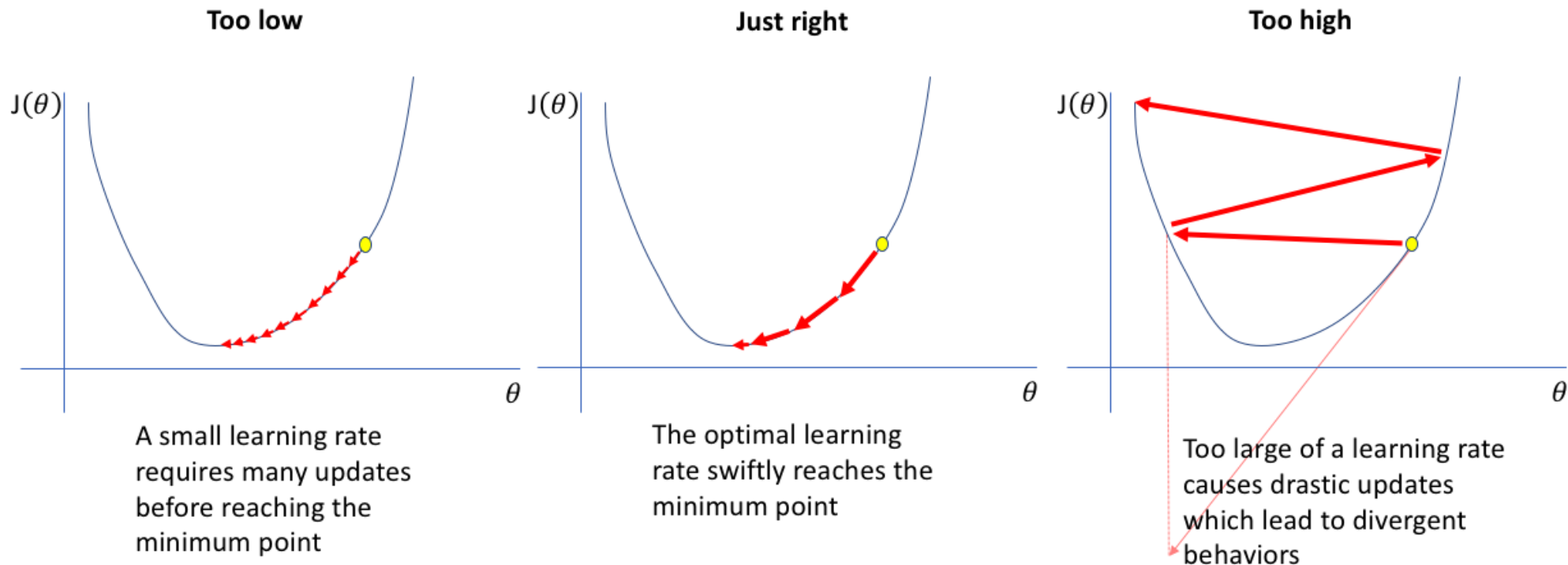
How can we set the
learning rate?

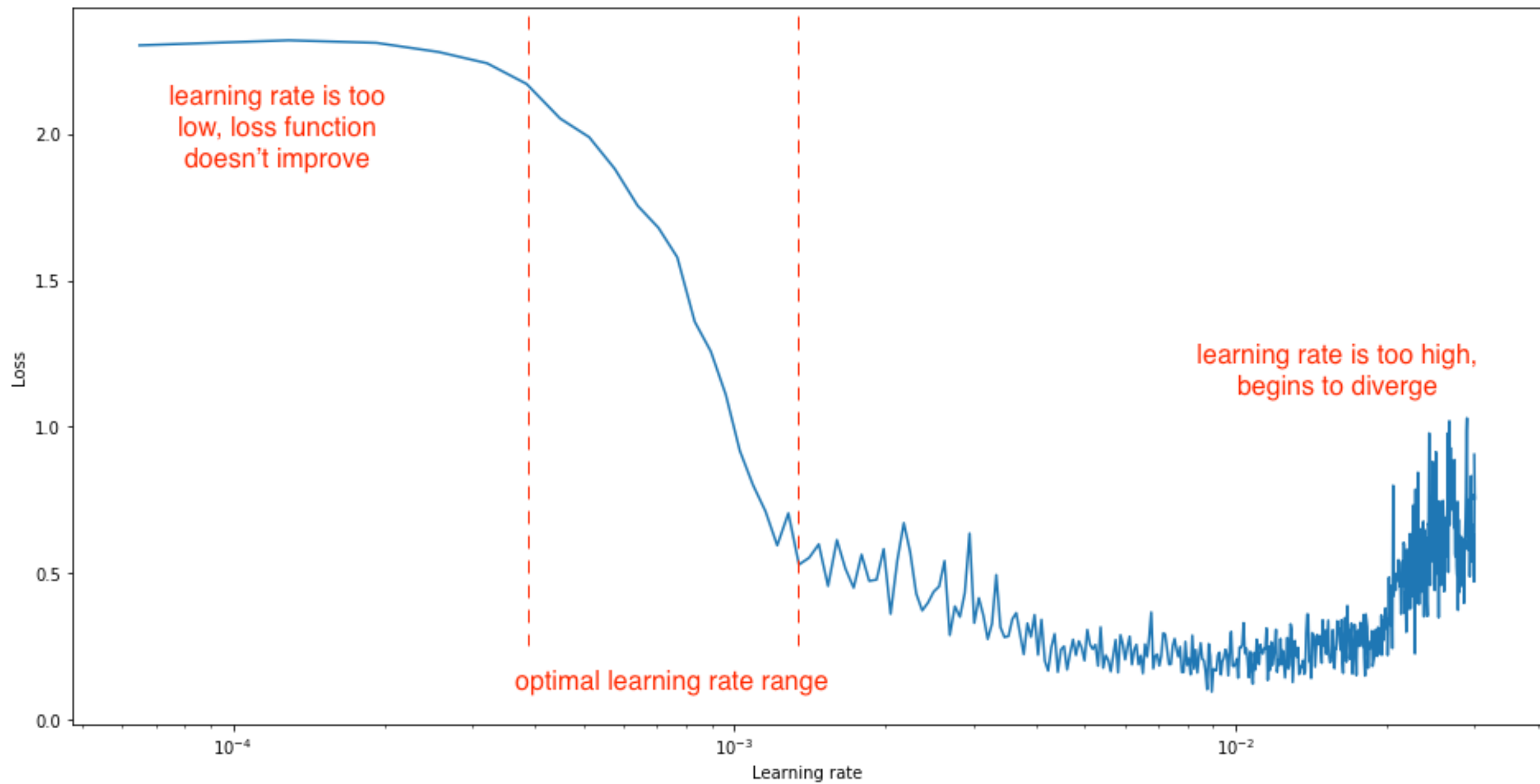
Learning Rate vs Loss

```
Epoch 1/20
2024-05-13 13:28:55.223274: I tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry.cc:117
219/219 [=====] - 3s 5ms/step - loss: 0.1955 - root_mean_squared_error: 0.4422
Epoch 2/20
219/219 [=====] - 1s 4ms/step - loss: 0.0552 - root_mean_squared_error: 0.2350
Epoch 3/20
219/219 [=====] - 1s 4ms/step - loss: 0.0309 - root_mean_squared_error: 0.1758
Epoch 4/20
219/219 [=====] - 1s 4ms/step - loss: 0.0208 - root_mean_squared_error: 0.1442
Epoch 5/20
219/219 [=====] - 1s 4ms/step - loss: 0.0157 - root_mean_squared_error: 0.1253
Epoch 6/20
219/219 [=====] - 1s 4ms/step - loss: 0.0129 - root_mean_squared_error: 0.1137
Epoch 7/20
219/219 [=====] - 1s 4ms/step - loss: 0.0113 - root_mean_squared_error: 0.1063
Epoch 8/20
219/219 [=====] - 1s 4ms/step - loss: 0.0101 - root_mean_squared_error: 0.1007
```

Setting the Learning rate

- **Small learning rate** converges slowly and gets stuck in false local minima
- **Large learning rates** overshoot, become unstable and diverge.
- **Stable learning rates** converge smoothly and avoid local minima





How to deal with this?

- Idea 1: Try lots of learning rates and see what works “just right”
- Idea 2: Do something smarter! Design a learning rate that adapts to the landscape

Adaptive Learning Rates

- Learning rates are no longer fixed
- Can be made larger or smaller depending on:
 - How large gradient is
 - How fast learning is happening
 - Size of particular weights
 - Etc...

Adaptive Learning Rate Algorithms

- **SGD** (Stochastic Gradient Descent) updates the model parameters based on the gradient of the loss function with respect to a **randomly selected subset of the training data at each iteration**.
- **AdaGrad** (Adaptive Gradient) is effective in scenarios where **data is sparse** or when dealing with features with **different scales**.
- **RMSProp** (Root Mean Squared Prop): also effective with sparse data or non-stationary objectives.
- **ADAM** (Adaptive Momentum Estimation) is an optimization algorithm that combines the advantages of both AdaGrad and RMSProp.
- **NADAM** combines the adaptive LR capabilities of ADAM with the momentum update rule of Nesterov.

Adaptive Learning Rate (Gradient Descent) Algorithms

Algorithm

- SGD
- Adam
- Adadelta
- Adagrad
- RMSProp

TF Implementation



```
tf.keras.optimizers.SGD
```



```
tf.keras.optimizers.Adam
```



```
tf.keras.optimizers.Adadelta
```



```
tf.keras.optimizers.Adagrad
```



```
tf.keras.optimizers.RMSProp
```

Reference

Kiefer & Wolfowitz. "Stochastic Estimation of the Maximum of a Regression Function." 1952.

Kingma et al. "Adam: A Method for Stochastic Optimization." 2014.

Zeiler et al. "ADADELTA: An Adaptive Learning Rate Method." 2012.

Duchi et al. "Adaptive Subgradient Methods for Online Learning and Stochastic Optimization." 2011.

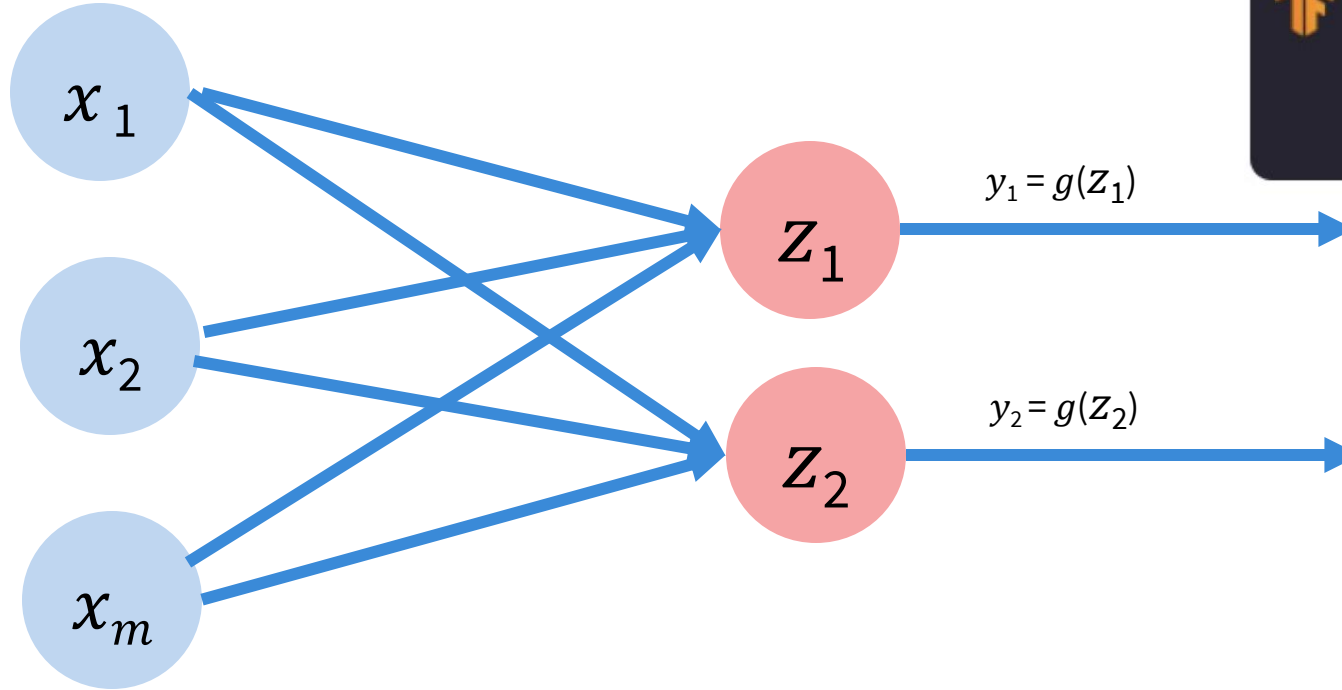


Dense layer from scratch

```
class MyDenseLayer(tf.keras.layers.Layer):  
    def __init__(self, input_dim, output_dim):  
        super(MyDenseLayer, self).__init__()  
  
        # initialise weights and bias  
        self.W = self.add_weight([input_dim, output_dim])  
        self.b = self.add_weight([1, output_dim])  
  
    def call(self, inputs):  
        # Forward propagate the inputs  
        z = tf.matmul(input, self.W) + self.b  
        # Feed through a non-linear activation  
        output = tf.math.sigmoid(z)  
  
        return output
```

Multi Output Perceptron

- Because all inputs are densely connected to all outputs, these layers are called **Dense** layers.



```
import tensorflow as tf  
  
layer = tf.keras.layers.Dense(  
    units=2)
```

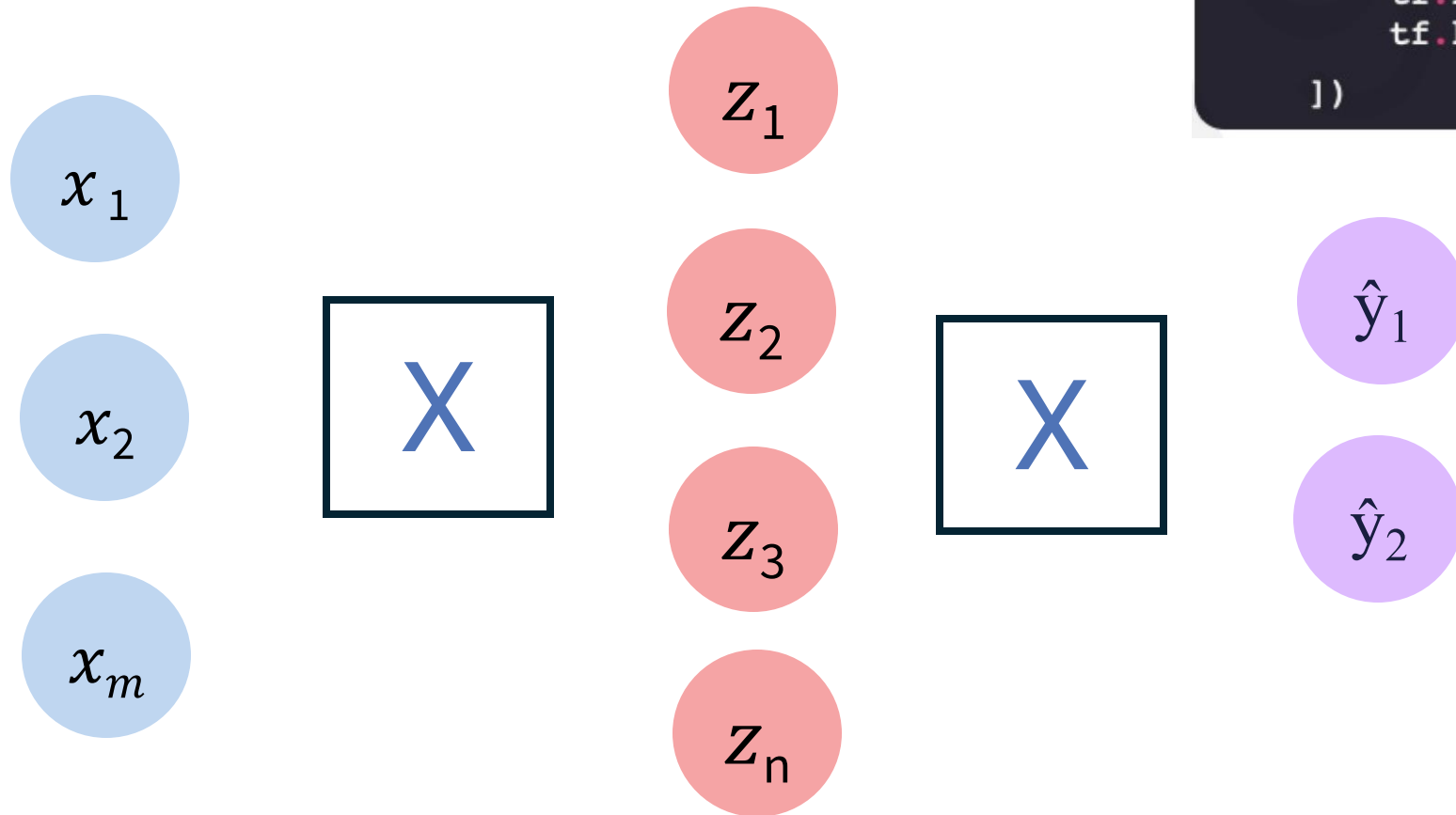
$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

Inputs (x)

Weights (w)

Output (y) = g (act. func of bias and sum)

Multi Output Perceptron



```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(n),
    tf.keras.layers.Dense(2)
])
```




Bringing everything together

```
import tensorflow as tf
model = tf.keras.Sequential([...])
# pick your favourite optimizer
optimizer = tf.keras.optimizer.SGD()
while True:  # loop forever
    # forward pass through the network
    prediction = model(x)
    with tf.GradientTape() as tape:
        # compute the loss
        loss = compute_loss(y, prediction)
    # update the weights using the gradient
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables)))
```

Neural Networks in Practice: Mini-batches

Gradient Descent

Algorithm

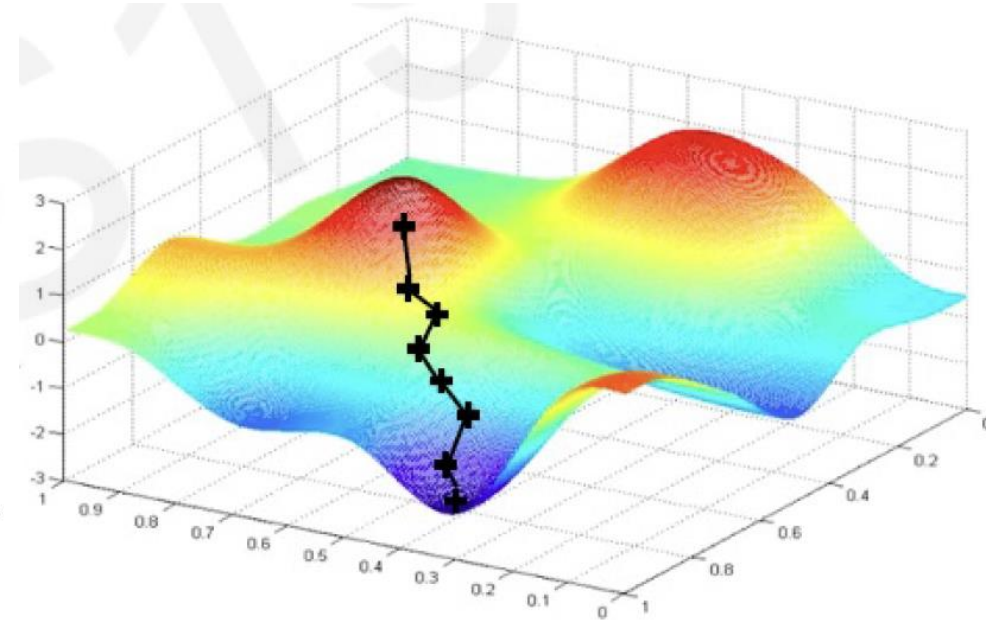
1. Initialise weights randomly
2. Loop until convergence:
3. Compute gradient
4. Update weights
5. Return weights

$$\sim \mathcal{N}(0, \sigma^2)$$

$$\frac{\partial J_i(W)}{\partial W}$$

$$W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$$

Can be computationally
intensive to compute!



Stochastic Gradient Descent

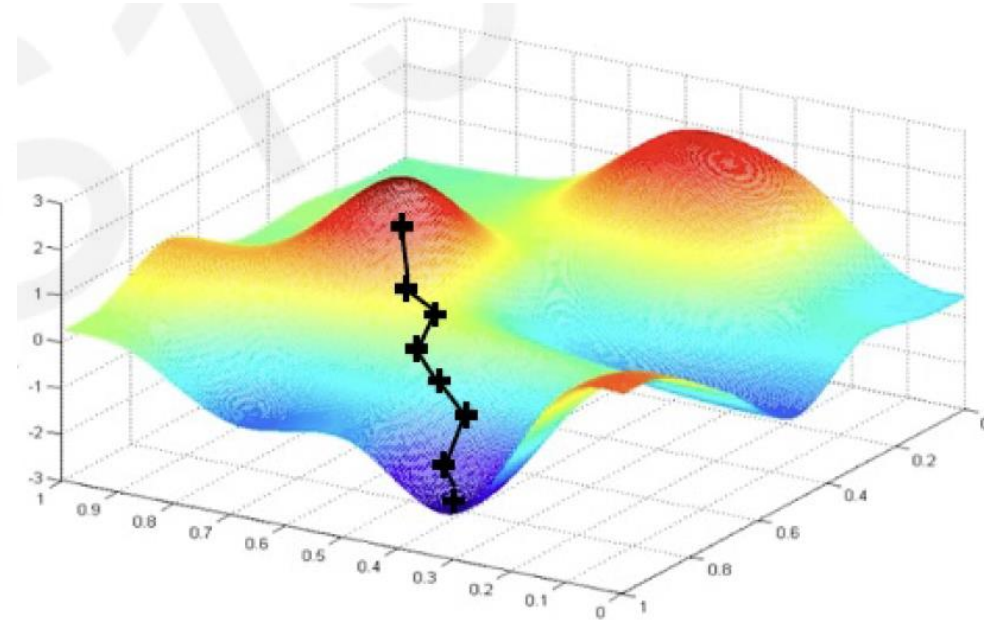
Algorithm

1. Initialise weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. **Pick data point i**
4. Compute gradient $\frac{\partial J_i(W)}{\partial W}$
5. Update weights $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
6. Return weights

$$\frac{\partial J_i(W)}{\partial W}$$

$$W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$$

Easy to compute but
very noisy (stochastic)!



Stochastic Gradient Descent

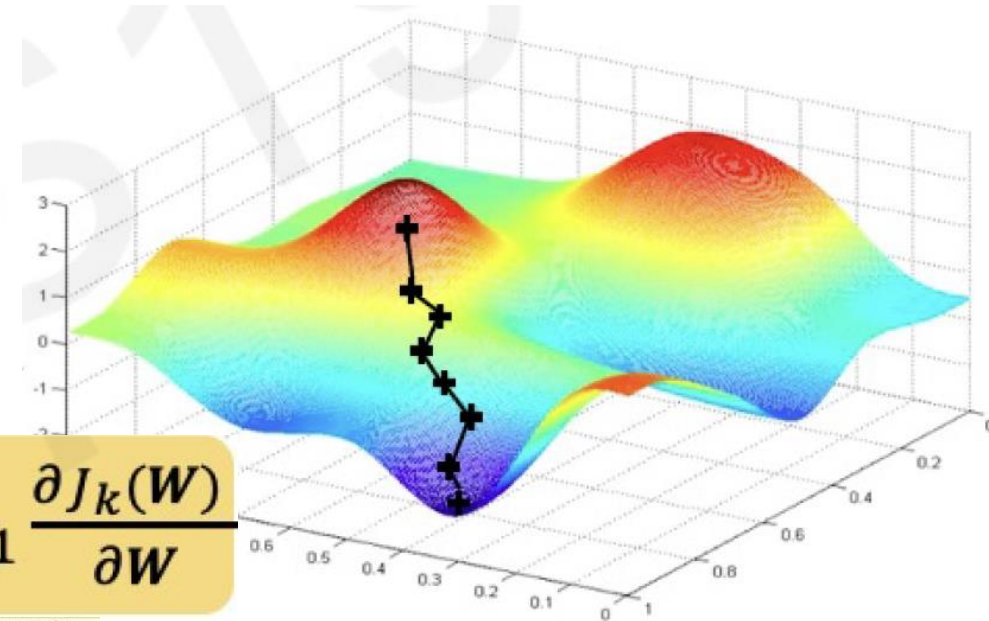
Algorithm

1. Initialise weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. **Pick batch of B data points**
4. Compute gradient
5. Update weights
6. Return weights

$$\frac{\partial J(W)}{\partial W} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(W)}{\partial W}$$

$$W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$$

Fast to compute and a much better estimate of the true gradient!



Mini-batches while training

More accurate estimation of gradient

Smoother convergence

Allows for larger learning rates

Mini-batches lead to fast training!

Can parallelize computation

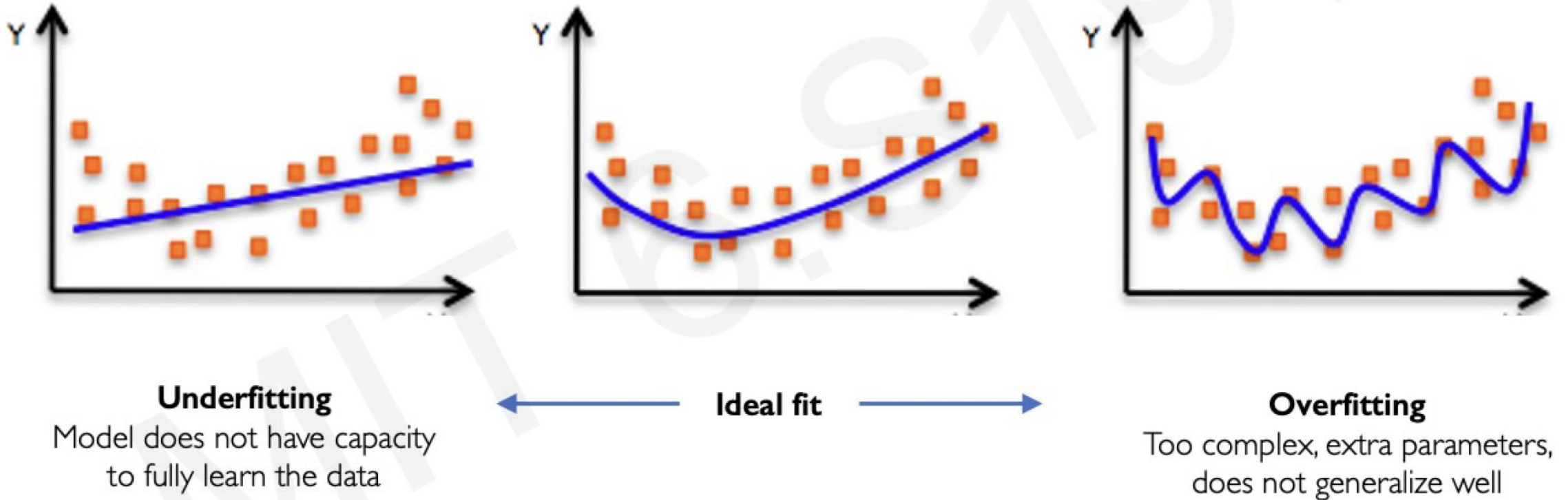
Achieve significant speed increases on GPUs

Batch vs Stochastic Gradient Descent

- **Batch gradient descent (BGD)** involves assessing the error for every example in the training dataset but holds off on updating the model until all examples have been processed.
- **Stochastic gradient descent (SGD)** entails both calculating the error and updating the model for each individual example in the training dataset.
- **Mini-Batch** gradient descent divides the training dataset into smaller batches, which are then utilized to compute model error and adjust model coefficients. This method, widely employed in deep learning, strikes a balance between BGD and SGD.

Neural Networks in Practice: Overfitting

The Problem of Overfitting



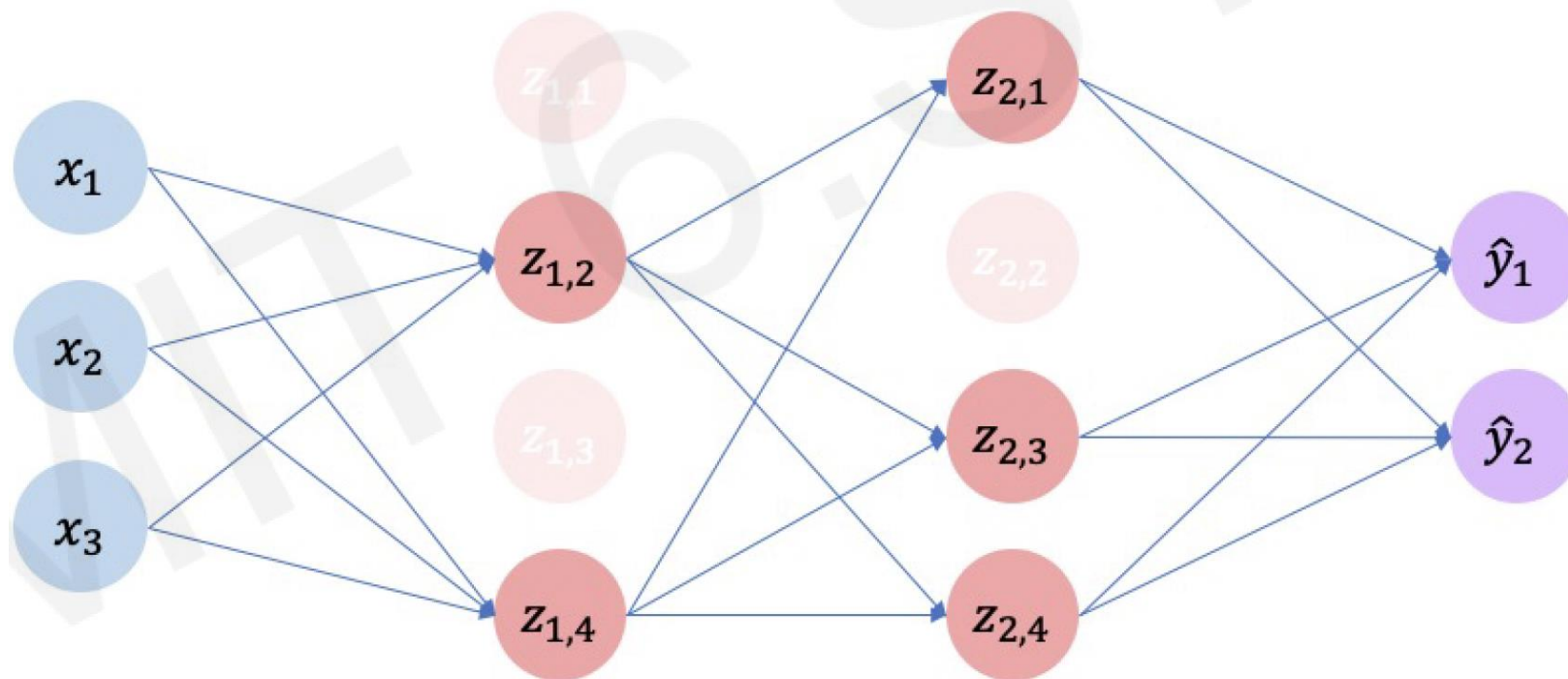
Regularization

- **What is it?**
- Technique that constrains our optimization problem to discourage complex models.
- **Why do we need it?**
- Improve generalization of our model on unseen data.

Regularization 1: Dropout

- During training, randomly set some activations to 0
 - Typically, 'drop' 50% of activations in layer
 - Forces network not to rely on any one node.

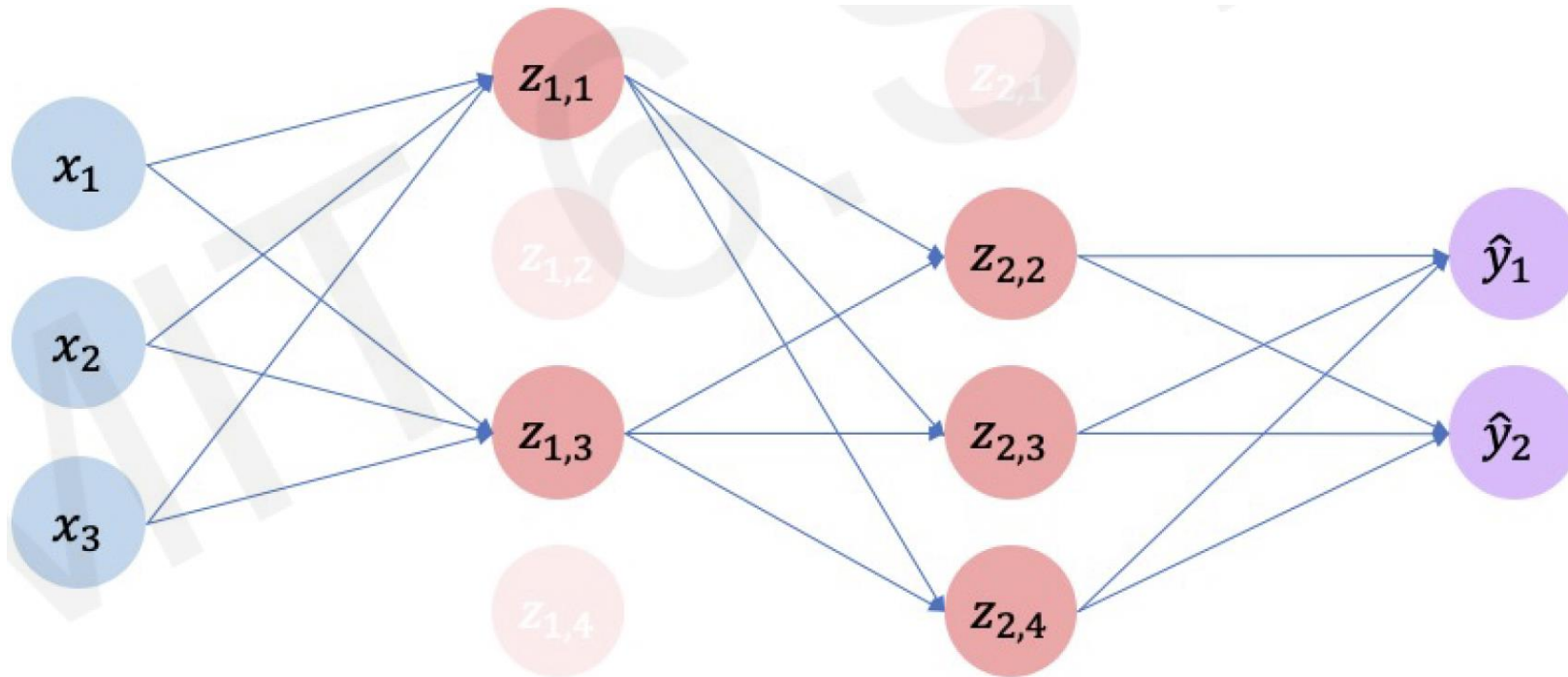
 `tf.keras.layers.Dropout(p=0.5)`



Regularization 1: Dropout

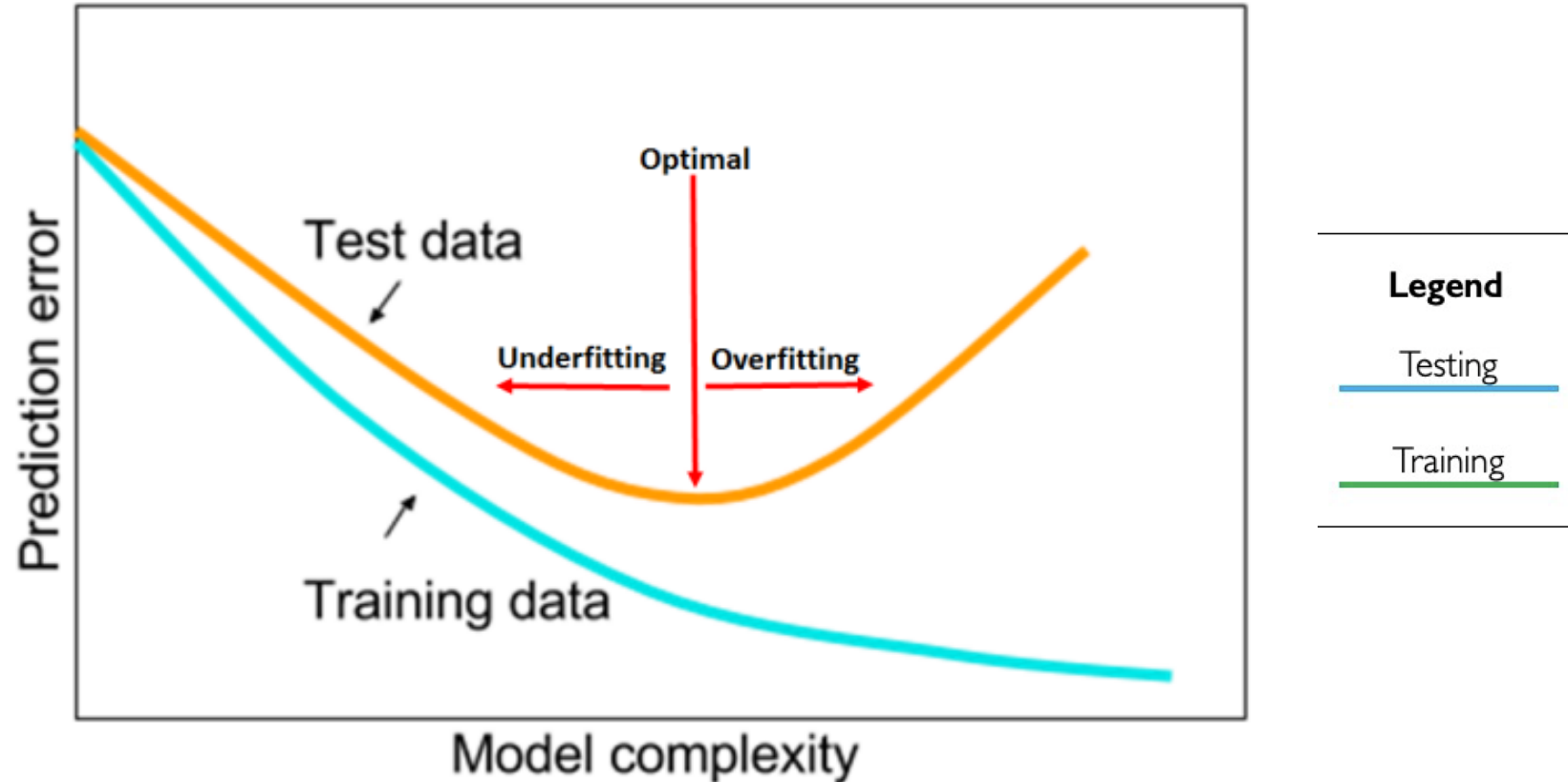
- During training, randomly set some activations to 0
 - Typically, 'drop' 50% of activations in layer
 - Forces network not to rely on any one node.

 `tf.keras.layers.Dropout(p=0.5)`



Regularization 2: Early Stopping

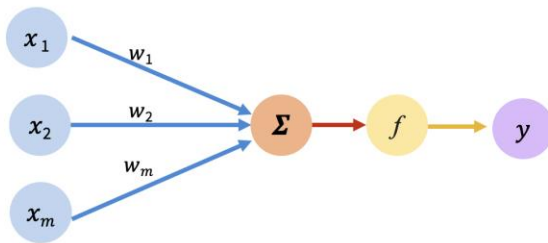
- Stop training before we have a chance to overfit.



Concepts Review

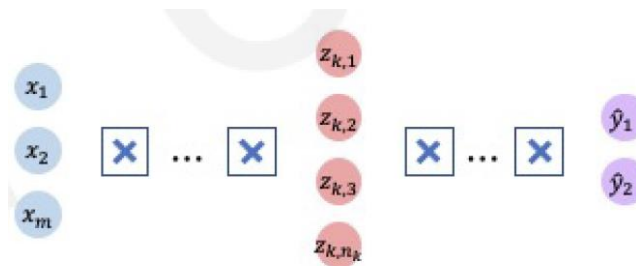
The Perceptron

- Structural building blocks
- Nonlinear activation functions



Neural Networks

- Stacking Perceptrons to form neural networks
- Optimization through backpropagation



Training in Practice

- Adaptive learning
- Batching
- Regularization

